Debre Markos University Bure Campus

Department of Computer Science

Data Structures and Algorithms Teaching Material

(CoSc 2092)

Prepared By: Robel Mamo (MSc)

March, 2022

Preface

This teaching material is prepared in support of the Data Structures and Algorithms Curriculum. The course focuses on the study of data structures, algorithms and program efficiency. The objectives of the course are: to introduce the most common data structures like stack, queue, linked list, to give alternate methods of data organization and representation, to enable students use the concepts related to Data Structures and Algorithms to solve real world problems, to practice Recursion, Sorting, and searching on the different data structures, and to implement the data structures with a chosen programming language.

Dear students, in chapter one you have been studied about Introduction to Data Structures, Abstract Data Types, Abstraction, Algorithms, Properties of Algorithm, Algorithm Analysis Concepts, Complexity Analysis, and Asymptotic Analysis.

In chapter two, Simple Sorting and Searching Algorithms, Insertion Sort, Selection Sort, Bubble Sort, Pointer Sort, Linear Search, and Binary Search Algorithms.

In chapter three, you have been studied about Review on Pointer and Dynamic Memory allocation, Singly Linked List and Its Implementation, Doubly Linked List and Its Implementation, and Circular Linked Lists and Its Implementation.

In chapter four, Stacks, Properties of Stack, Array Implementation of Stack, Linked List Implementation of Stack, and Application of Stack

Chapter five is about Queue, Properties of Queue, Array Implementation of Queue, Linked List Implementation of Queue, Double Ended Queue (Deque), Priority Queue, and Application of Queues.

Chapter six is about Trees, Binary Tree and Binary Search Trees, Basic Tree Operations, Traversing in a Binary Tree, and General Trees and Their Implementations.

In chapter seven,Introduction to Graphs, Directed vs Undirected Graph, and Traversing Graph.

Chapter eight is about Advanced Sorting and Searching algorithms, Shell Sort, Quick Sort, Heap Sort, Merge Sort, and Hashing.

Table of Contents

Chapter One: Introduction to Data Structures and Algorithms

1.1. Introduction to Data Structures

The study of data structures, a fundamental component of a computer science education, serves as the foundation upon which many other computer science fields are built. Knowledge of data structures is a must for students who wish to do work in software development.

A data structure is a way of storing data in a computer so that it can be used efficiently. It is the specification of the elements of the structure, the relationships between them, and the operation that may be performed upon them. Data Structures is about rendering data elements in terms of some relationship, for better organization and storage. It should be designed and implemented in such a way that it reduces the complexity and increases the efficiency.

Consider as an example books in a library. It can be stored in a shelf arbitrarily or using some defined orders such as sorted by Title, Author and so on. It can be said that the study of data structures involves the storage, retrieval and manipulation of information.

The various operations that can be performed over Data Structures are:

- Create
- Retrieve
- Update
- Delete etc.

Activity 1.1

- ✓ Define data structure?
- ✓ Explain how a data structure should be designed?

Types of Data Structures

Primitive Data Structures: As we have discussed above, anything that can store data can be called as a data structure, hence Integer, Float, Boolean, Char etc, all are data structures. They are known as Primitive Data Structures.

Non-Primitive Data Structure: Then we also have some complex Data Structures, which are used to store large and connected data. They are those that can be obtained from the primitive data structures. Some examples of these data structure are: arrays, stacks, queues, linked lists, Trees, Graph, etc.

If the allocation of memory is sequential or continuous then such a data structure is called a non-primitive linear structure. In linear data structure, single level is involved. Therefore, we can traverse all the elements in single run only. Linear data structures are easy to implement because computer memory is arranged in a linear way. Example: arrays, stacks, queues, linked lists, etc.

Array

The array is a type of data structure that stores elements of the same type. These are the most basic and fundamental data structures. Data stored in each position of an array is given a positive value called the index of the element. The index helps in identifying the location of the elements in an array.If supposedly we have to store some data i.e. the price of ten cars, then we can create a structure of an array and store all the integers together. This doesn't need creating ten separate integer variables. Therefore, the lines in a code are reduced and memory is saved. The index value starts with 0 for the first element in the case of an array.

Stack

The data structure follows the rule of LIFO (Last In-First Out) where the data last added element is removed first. Push operation is used for adding an element of data on a stack and the pop operation is used for deleting the data from the stack. This can be explained by the example of books stacked together. In order to access the last book, all the books placed on top of the last book have to be safely removed.

Queue

This structure is almost similar to the stack as the data is stored sequentially. The difference is that the queue data structure follows FIFO which is the rule of First In-First Out where the first added element is to exit the queue first. Front and rear are the two terms to be used in a queue. Enqueue is the insertion operation and dequeue is the deletion operation. The former is performed at the end of the queue and the latter is performed at the start end. The data structure

might be explained with the example of people queuing up to ride a bus. The first person in the line will get the chance to exit the queue while the last person will be the last to exit.

LinkedList

Linked lists are the types where the data is stored in the form of nodes which consist of an element of data and a pointer. The use of the pointer is that it points or directs to the node which is next to the element in the sequence. The data stored in a linked list might be of any form, strings, numbers, or characters. Both sorted and unsorted data can be stored in a linked list along with unique or duplicate elements.

If the allocation of memory is not sequential but random or discontinuous then such a data structure is called a non-primitive non-linear structure. In a non-linear data structure, single level is not involved. Therefore, we can't traverse all the elements in single run only. Non-linear data structures are not easy to implement in comparison to linear data structure. It utilizes computer memory efficiently in comparison to a linear data structure. Example, Trees and Graphs

Tree

A tree data structure consists of various nodes linked together. The structure of a tree is hierarchical that forms a relationship like that of the parent and a child. The structure of the tree is formed in a way that there is one connection for every parent-child node relationship. Only one path should exist between the root to a node in the tree. Various types of trees are present based on their structures like binary tree, binary search tree, etc.

Graph

Graphs are those types of non-linear data structures which consist of a definite quantity of vertices and edges. The vertices or the nodes are involved in storing data and the edges show the vertices relationship. The difference between a graph to a tree is that in a graph there are no specific rules for the connection of nodes. Real-life problems like social networks, telephone networks, etc. can be represented through the graphs.

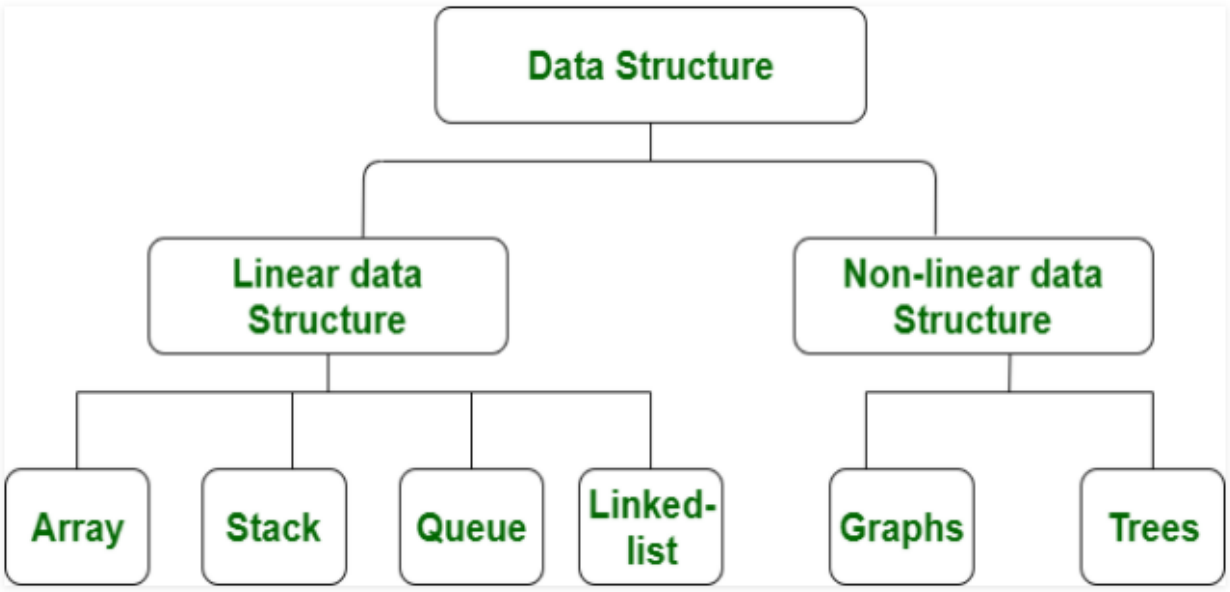


Fig. 1.1 Types of data structures

S.NO	Linear Data Structure	Non-linear Data Structure
1.	In a linear data structure, data elements are arranged in a linear order where each and every elements are attached to its previous and next adjacent.	In a non-linear data structure, data elements are attached in hierarchically manner.
2.	In linear data structure, single level is involved.	Whereas in non-linear data structure, multiple levels are involved.
3.	Its implementation is easy in comparison to non-linear data structure.	While its implementation is complex in comparison to linear data structure.
4.	In linear data structure, data elements can be traversed in a single run only.	While in non-linear data structure, data elements can't be traversed in a single run only.
5.	In a linear data structure, memory is not utilized in an efficient way.	While in a non-linear data structure, memory is utilized in an efficientway.

S.NO	Linear Data Structure	Non-linear Data Structure
6.	Its examples are: array, stack, queue, linked list, etc.	While its examples are: trees and graphs.
7.	Applications of linear data structures are mainly in application software development.	Applications of non-linear data structures are in Artificial Intelligence and image processing.

Table 1.1 Linear vs. Non-linear structures

All these data structures allow us to perform different operations on data. We select these data structures based on which type of operation is required.

The data structures can also be classified on the basis of the following characteristics:

Homogeneous: In homogeneous data structures, all the elements are of same type. Example: Array

Non-Homogeneous: In Non-Homogeneous data structure, the elements may or may not be of the same type. Example: Structures

Static: Static data structures are those whose sizes and structures associated memory locations are fixed, at compile time. The content of the data structure can be modified but without changing the memory space allocated to it. Example: Array

Dynamic: Dynamic structures are those which expands or shrinks depending upon the program need and its execution. Also, their associated memory locations changes. In Dynamic data structure the size of the structure is not fixed and can be modified during the operations performed on it. Dynamic data structures are designed to facilitate change of data structures in the run time. Example: Linked List

Static Data Structure vs Dynamic Data Structure

Static Data structure has fixed memory size whereas in Dynamic Data Structure, the size can be randomly updated during run time which may be considered efficient with respect to memory

complexity of the code. Static Data Structure provides easier access to elements with respect to dynamic data structure. Unlike static data structures, dynamic data structures are flexible.

Activity 1.2

- ✓ Explain primitive and non-primitive data structures with example?
- ✓ Explain the difference between linear and non-linear, homogeneous and non-homogeneous, and static and dynamic data structures?

1.1.1. Abstract Data Type

Abstract Data Type: the combination of data together with its methods or operations is called an Abstract Data Type (ADT).The operations may act upon an item by modifying it, searching for some details in it, or storing something in it and so on. The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented. It does not specify how data will be organized in memory and what algorithms will be used for implementing the operations. It is called "abstract" because it gives an implementation-independent view. The process of providing only the essentials and hiding the details is known as abstraction.Think of ADT as a black box which hides the inner structure and design of the data type.

After the operations are precisely specified, the implementation of the program may start. The implementation decides which data structure should be used to make execution most efficient in terms of time and space.

1.1.2. Data Abstraction

Data abstraction: describes what information is stored in without being specific about how the information is stored or organized. The advantage of data abstraction is that a designer can start with what data will be used, and postpone implementation decisions.

Activity 1.3

- ✓ Explain the need for data abstraction?
- ✓ Define ADT?

1.2. Algorithms

Algorithm is a step by step procedure to solve a problem.is a finite set of instructions or logic, written in order, to accomplish a certain predefined task. Algorithm is not the complete code or program, it is just the core logic(solution) of a problem, which can be expressed either as an informal high level description as pseudocode or using a flowchart.

The purpose of an algorithm is to accept input values, to change a value hold by a data structure, to re-organize the data structure itself (e.g. sorting), to display the content of the data structure, and so on.More than one algorithm is possible for the same task and we need select the right one that can run efficiently. An algorithm is said to be efficient and fast, if it takes less time to execute and consumes less memory space. The performance of an algorithm is measured on the basis of following properties:

- Time Complexity
- Space Complexity

Space Complexity: It is the amount of memory space required by the algorithm, during the course of its execution. Space complexity must be taken seriously for multi-user systems and in situations where limited memory is available.An algorithm generally requires space for following components:

- **Instruction Space**: It is the space required to store the executable version of the program. This space is fixed, but varies depending upon the number of lines of code in the program.
- **Data Space**: It is the space required to store all the constants and variables(including temporary variables) value.
- **Environment Space**: It is the space required to store the environment information needed to resume the suspended function.

Time Complexity: Time Complexity is a way to represent the amount of time required by the program to run till its completion. It's generally a good practice to try to keep the time required minimum, so that our algorithm completes it's execution in the minimum time possible.

1.2.1. Properties of Algorithms

- **Finiteness:** any algorithm should have finite number of steps to be followed.
- **Absence of ambiguity:** the algorithm should have one and only one interpretation during execution.
- **Sequential:** it should have a start step and a halt step for a final result
- **Feasibility:** the possibility of each algorithm to be executed.
- **Correctness**: Every step of the algorithm must generate a correct output.
- **Input/Output**: zero or more input, one or more output. And so on.

Activity 1.4

- ✓ Define algorithm?
- ✓ What is the purpose of an algorithm?

1.2.2. Algorithm Analysis Concepts

The same problem can frequently be solved with algorithms that differ in efficiency. Efficiency difference may not be noticeable for small data, but become very important for large amounts of data.

1.2.3. Complexity analysis

It is a measurement that indicates how much effort is needed to apply an algorithm or how costly it is. The two efficiency criteria are time and space. The factor of time is more important than that of space, so efficiency considerations usually focus on the amount of time elapsed when processing data. When it comes to analyzing the complexity of any algorithm in terms of time and space, we can never provide an exact number to define the time required and the space required by the algorithm, instead we express it using some standard notations.

Complexity analysis is used to compare the efficiency of algorithms.To evaluate an algorithm's efficiency, real-time units such as microseconds and nanoseconds should not be used. Rather, logical units that express a relationship between the size n of a file and the amount of time t required to process the data should be used.

Empirical Complexity Analysis: in this method the total running time of the program is considered. It uses the system time to calculate the running time of the program. Empirical analysis cannot be used for measuring efficiency of algorithms. This is because the total running time of the program algorithm varies on the processor speed, current processor load, input size of the given algorithm and software environment.

Asymptotic Complexity Analysis: consider $t=T(n)=n^2+5n$; for all $n>5$, n^2 is largest, and for every large n , the 5n term is insignificant. Therefore we can approximate T(n) by the n^2 term only. This is called asymptotic complexity.It is used when it is difficult or unnecessary to determine true computational complexity and usually it is difficult/impossible to determine true computational complexity. So asymptotic complexity is the most common measure for algorithm analysis.

Let us take another example, if some algorithm has a time complexity of $T(n) = (n^2 + 3n + 4)$, for large values of n, the 3n + 4 part will become insignificant compared to the n^2 part.

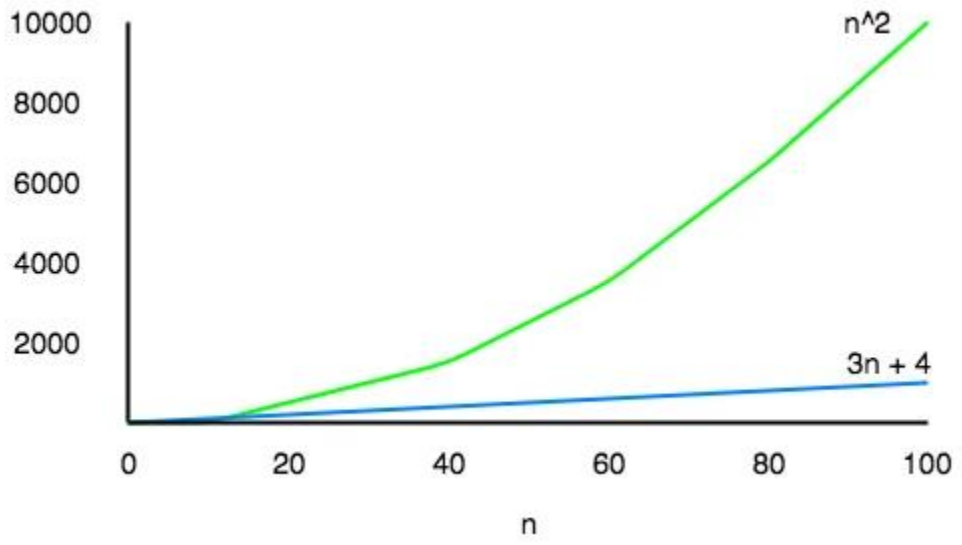


Fig. 1.2 Asymptotic complexity analysis

For n = 1000, n^2 will be 1000000 while 3n + 4 will be 3004.

Also, when we compare the execution times of two algorithms the constant coefficients of higher order terms are also neglected.

An algorithm that takes a time of $200n^2$ will be faster than some other algorithm that takes n^3 time, for any value of n larger than 200. Since we're only interested in the asymptotic behavior of the growth of the function, the constant factor can be ignored too.

Activity 1.5

- ✓ Define computational complexity analysis?
- ✓ What is the difference between empirical and asymptotic complexity analysis?

1.3. Asymptotic Analysis

The word Asymptotic means approaching a value or curve arbitrarily closely (i.e., as some sort of limit is taken).Remember studying about Limits in High School, this is the same. The only difference being, here we do not have to find the value of any expression where n is approaching any finite number or infinity, but in case of Asymptotic notations, we use the same model to ignore the constant factors and insignificant parts of an expression, to device a better way of representing complexities of algorithms, in a single coefficient, so that comparison between algorithms can be done easily.

Let's take an example to understand this:

If we have two algorithms with the following expressions representing the time required by them for execution, then:

Expression 1: $(20n^2 + 3n - 4)$

Expression 2: $(n^3 + 100n - 2)$

Now, as per asymptotic notations, we should just worry about how the function will grow as the value of n (input) will grow, and that will entirely depend on n^2 for the Expression 1, and on n^3 for Expression 2. Hence, we can clearly say that the algorithm for which running time is represented by the Expression 2, will grow faster than the other one, simply by analysing the highest power coefficient and ignoring the other constants (20 in $20n^2$) and insignificant parts of the expression($3n - 4$ and $100n - 2$).

The main idea behind casting aside the less important part is to make things manageable, since we are not dealing with real time.All we need to do is, first analyse the algorithm to find out an expression to define it's time requirements and then analyse how that expression will grow as the input(n) will grow.

How do we estimate the complexity of algorithms?

Space Complexity of Algorithms

Whenever a solution to a problem is written some memory is required to complete. For any algorithm memory may be used for the following:

- Variables (This include the constant values, temporary values)
- Program Instruction
- Execution

Space complexity is the amount of memory used by the algorithm (including the input values to the algorithm) to execute and produce the result.

Sometime Auxiliary Space is confused with Space Complexity. But Auxiliary Space is the extra space or the temporary space used by the algorithm during its execution.

Space Complexity = Auxiliary Space + Input space

When calculating the Space Complexity of any algorithm, we usually consider only Data Space and we neglect the Instruction Space and Environmental Stack.

Calculating the Space Complexity

For calculating the space complexity, we need to know the value of memory used by different type of data type variables, which generally varies for different operating systems, but the method for calculating the space complexity remains the same.

Type	Size
bool, char, unsigned char, signed char, __int8	1 byte
__int16, short, unsigned short, wchar_t, __wchar_t	2 bytes
float, __int32, int, unsigned int, long, unsigned long	4 bytes
double, __int64, long double, long long	8 bytes

Table 1.2 Data types and their size

Now let's take an example:

```
{
   int z = a + b + c;
   return(z);
}
```

In the above expression, variables a, b, c and z are all integer types, hence they will take up 4 bytes each, so total memory requirement will be (4(4) + 4) = 20 bytes, this additional 4 bytes is for return value. And because this space requirement is fixed for the above example, hence it is called Constant Space Complexity.

Let's have another example:

```
// n is the length of array a[]
int sum(int a[], int n)
{
        int x = 0;                  // 4 bytes for x
        for(int i = 0; i < n; i++)          // 4 bytes for i
        {
           x  = x + a[i];
}
```

In the above code, 4*n bytes of space is required for the array a[] elements. 4 bytes each for x, n, i and the return value.Hence the total memory requirement will be (4n + 12), which is

increasing linearly with the increase in the input value n, hence it is called as Linear Space Complexity.

Similarly, we can have quadratic and other complex space complexity as well, as the complexity of an algorithm increases.But we should always focus on writing algorithm code in such a way that we keep the space complexity minimum.

Activity 1.6

- ✓ Define space complexity?
- ✓ How do we calculate space complexity?

Time Complexity of Algorithms

Time complexity of an algorithm signifies the total time required by the program to run till its completion. For Any problem which must be solved using a program, there can be infinite number of solutions. Let's take a simple example to understand this. Below we have two different algorithms to find square of a number (for some time, forget that square of any number n is n*n):

One solution to this problem can be, running a loop for n times, starting with the number n and adding n to it, every time.

```
/* we have to calculate the square of n*/
for i=1 to n
   do n = n + n
// when the loop ends n will hold its square
return n
```

Or, we can simply use a mathematical operator * to find the square.

```
/* we have to calculate the square of n*/
return n*n
```

In the above two simple algorithms, you saw how a single problem can have many solutions. While the first solution required a loop which will execute for n number of times, the second

solution used a mathematical operator * to return the result in one line. So which one is the better approach, of course the second one.

Time Complexity is most commonly estimated by counting the number of elementary steps performed by any algorithm to finish execution. Like in the example above, for the first code the loop will run n number of times, so the time complexity will be n atleast and as the value of n will increase the time taken will also increase. While for the second code, time complexity is constant, because it will never be dependent on the value of n, it will always give the result in 1 step.

And since the algorithm's performance may vary with different types of input data, hence for an algorithm we usually use the worst-case Time complexity of an algorithm because that is the maximum time taken for any input size.

Calculating Time Complexity

We use algorithm analysis rule i.e. Algorithm analysis – Find f(n)- helps to determine the complexity of an algorithm

Order of Magnitude: g(n) belongs to O(f(n)) – helps to determine the category of the complexity to which it belongs.

Analysis Rule

1. Assume an arbitrary time unit
2. Execution of one of the following operations takes time unit 1
    - Assignment statement Eg. Sum=0;
    - Single I/O statement E.g. cin>>sum; cout<<sum;
    - Single Boolean statement E.g. !4;
    - Single arithmetic E.g. a+b;
    - Function return E.g. return(sum);
3. Selection statement
    - Time for condition evaluation + the maximum time of its clauses
4. Loop statement

- Time for loop assignment + time for loop comparison + time for loop iteration + time for loop body multiplied by the number of iteration of the loop

5. For function call

- 1+ time(parameters) + body time

Calculate T(n) for the following

```
k=0;
cout<<“enter an integer”;
cin>>n;
    For (int i=0;i<n;i++)
    K++;
```

Solution:

T(n) = 1+1+1+ (1+n+1+ n +n)=5+3n

```
 i=0;
while (i<n)
      {
x++;
        i++;
}
j=1;
    while(j<=10)
      {
x++;
       j++;
}
```

Solution:

T(n)=1+n+1+n+n+1+11+10+10=3n+34

```
for(i=1;i<=n;i++)
        for(j=1;j<=n; j++)
              k++;
```

Solution:

$T(n)=1+n+1+n+n(1+n+1+n+n)=3n^2+4n+2$

```
Sum=0;
      if(test==1)
       {
        for (i=1;i<=n;i++)
        sum=sum+i
       }
      else
      {
        cout<<sum;
      }
```

Solution:

$T(n)=1+1+Max(1+n+1+n+n+n,1)= 4n+4$

Activity 1.7

- ✓ Define time complexity?
- ✓ How do we calculate time complexity?

Algorithm Analysis Categories

Algorithm must be examined under different situations to correctly determine their efficiency for accurate comparisons

Best Case Analysis:Assumes that data are arranged in the most advantageous order. It also assumes the minimum input size.

E.g. For sorting: the best case is if the data are arranged in the required order. For searching: the required item is found at the first position.

Note: Best Case computes the lower boundary of T(n).

It causes fewest numbers of executions

Worst Case Analysis:Assumes that data are arranged in the disadvantageous order. It also assumes that the maximum input size.

Eg. For sorting: data are arranged in opposite required order. For searching: the required item is found at the end of the item or the item is missing

It computes the upper bound of T(n).

It causes maximum number of execution.

Average Case Analysis: Assumes that data are found in random order. It also assumes random or average input size.

E.g. For sorting: data are in random order. For searching: the required item is found at any position.

It computes optimal bound of T(n)

It also causes average number of execution.

Best case and average case cannot be used to estimate (determine) complexity of algorithms

Worst case is the best to determine the complexity of algorithms.

Activity 1.8

- ✓ What mean by algorithm analysis category?
- ✓ Explain algorithm analysis categories?

Order of Magnitude

Order Of Magnitude refers to the rate at which the storage or time grows as function of problem size (function (n)).

It is expressed in terms of its relationship to some known functions – asymptotic analysis.

Types of Asymptotic Notations

1. Big-O Notation
2. Big – Omega (Ω)
3. Big –Theta (Θ)
4. Little o (small o)

Big-O Notation

Definition : The function T(n) is O(F(n)) if there exist constants c and N such that $T(n) \leq c.F(n)$ for all $n \geq N$.

As n increases, T(n) grows no faster than F(n) or in the long run (for large n) T grows at most as fast as F.

It computes the tight upper bound of T(n).

It describes the worst case analysis.

Example

Find F(n) such that T(n) = O(F(n)) for T(n) = 3n+5

Solution: $3n+5 \leq cn$

c=6, N=2

F(n)=n because c=6 for $3n+5 \leq 6n$, for All n>=2

T(n) = O(n)

Find F(n) such that $T(n) = n^2 + 5n$

$T(n) \leq c.F(n)$

$n^2 + 5n \leq c.\ n^2$

$1 + (5/n) \leq c$

$F(n) = n^2$

$T(n) = O(n^2)$

Therefore if we choose N=5, then c= 2; if we choose N=6, then c=1.83, and so on.So what are the 'correct' values for c and N ? The answer to this question, it should be determined for which value of N a particular term in T(n) becomes the largest and stays the largest.In the above example, the n^2 term becomes larger than the 5n term at n>5, so N=5, c=2 is a good choice.

Activity 1.9

- ✓ What mean by order of magnitude?
- ✓ Explain Big-O notation?

Chapter One Review Questions

1. What constitute a data structure?
2. Mention real-world problems that we can apply data structure?
3. Discuss the difference between array and linked list data structures?
4. Discuss the difference between tree and graph data structures?
5. Discuss the activities carried out during data abstraction, ADT, and implementation?
6. Discuss about algorithm performance
7. Discuss properties of algorithm
8. Discuss the rules used for algorithm analysis
9. Group assignment: Read and write about the following notations
    - Big – Omega (Ω)
    - Big –Theta (Θ)
    - Little o (small o)

Chapter Two: Simple Sorting and Searching Algorithms

Simple Sorting and Searching Algorithms: They are the most common & useful tasks in any software development. Example:

- Searching documents over the internet
- Searching files and folders in a hard disk
- Sorting students by their name, year and so on
- Sorting file, search results by file name, date created and so on

2.1. Sorting Algorithms

Sorting is nothing but arranging the data in ascending or descending order. The term sorting came into picture, as humans realized the importance of searching quickly.There are so many things in our real life that we need to search for, like a particular record in database, roll numbers in merit list, a particular telephone number in telephone directory, a particular page in a book etc. All this would have been a mess if the data was kept unordered and unsorted, but fortunately the concept of sorting came into existence, making it easier for everyone to arrange data in an order, hence making it easier to search. Sorting arranges data in a sequence which makes searching easier.

Sorting algorithms commonly consist of two types of operation: comparisons and data movements. A comparison is simply comparing one value in a list with another, and a data movement (swapping) is an assignment.There are many different techniques available for sorting, differentiated by their efficiency and space requirements. Following are some sorting techniques which we will be covering:

- Insertion sort
- Selection sort
- Bubble sort
- Pointer sort

Activity 2.1

- ✓ What are sorting algorithms?
- ✓ Explain the operations a sorting algorithm consists of?

2.1.1. Insertion Sort

As each element in the list is examined it is put into its proper place among the elements that have been examined. When the last element is put into its proper position the list is sorted and the algorithm is done.

Implementation

Swapping algorithm: it is the algorithm that interchanges the value of two variables. Swapping should be done by address and not by value to make the change permanent. If swapping is done by value, the change will have effect only in the current block of code.

```
void swap( dataType &x, dataType &y)
  {
    dataType temp;
    temp=x;
    x=y;
    y=temp;
  }
```

Comparison algorithm: it used to compare the values of two data elements to decide swapping is required or not between them. The algorithm starts the outer for loop from the second data element which is at index 1. A Boolean variable inserted is initialized as false and used to control swapping requirement between the data elements. For the inner for loop we will use while loop to check the value of j >=1 and inserted==false. If the conditions are satisfied the algorithm compares the second data element with the first data element and call the swap function if so. Otherwise it will set inserted true and decrement j for the next iteration. Note that inserted is true means there is no order problem between the examined data elements. The algorithm continues like this up to the comparison of the last data element and its previous data element and the last data element will be compared to its entire previous element if it is not inserted in its proper position. The outer for loop uses increment operator and inner while loop uses decrement operator for iteration.

```
for (i=1;i<=n-1;i++)
   {
     inserted =false;
```

```
      j=i;
    while ((j>=1) && (inserted == false))
        {
         If (dataElement[j]<dataElement[j-1])
            swap(dataElement[j],dataElement[j-1]);
         else
            inserted=true;
            j--
        }
     }
```

Example: write full program that can sort (5, 2, 3, 8, 1) using insertion sort algorithm

Solution:

```
#include<iostream.h>
void swap(int &x, int &y)          //& means by address
    {
     int temp;
     temp=x;
    x=y;
    y=temp;
   }
int main()
  {
    int a[5]={5, 2, 3, 8, 1};         // transform the unsorted data into array of size 5
    for(int i=1;i<=4;i++)
       {
        bool inserted=false;
        int j=i;
       while((j>=1) && (inserted==false))
           {
            if(a[j]<a[j-1])
          {
```

```
            swap(a[j],a[j-1]);
          }
         else
         {
           inserted=true;
         }
           j--;
        }
    }
  for(int k=0;k<5;k++)       //display the sorted data
    cout<<a[k];
    return 0;
}
/*output
1 2 3 5 8
*/
```

Big-O analysis of Insertion Sort Algorithm

Analysis involves number of comparisons and number of swaps. In the first pass there is 1 comparison and 1 swap. In the second pass there are 2 comparisons and 2 swaps and so on. In the $(n-1)^{th}$ pass there are n-1 comparisons and n-1 swaps. Totally there are n(n-1)/2 comparisons and n(n-1)/2 swaps. Therefore Big-O of insertion sort algorithm is n(n-1)/2 + n(n-1)/2 =$O(n^2)$.

Activity 2.2

- ✓ Explain how insertion sort algorithm works?
- ✓ Show in figure the iteration of the above insertion sort program?

2.1.2. Selection Sort

The algorithm finds the minimum element of the sub array and swaps it with the pivot elements.

Implementation

Swapping algorithm: The swapping algorithm used in selection sort algorithm is the same as the swapping algorithm that we have seen in insertion sort algorithm.

Comparison algorithm: it used to compare the values of two data elements to decide swapping is required or not between them. The algorithm assumes the first data element as minimum and hence the outer for loop starts i from 0 and sets it to be minimum index. The inner for loop starts j from i+1 and compare it with the data element found at minimum index. If the data element at j is less than the data element at minimum index then j will be minimum index. The inner for loop continues finding the exact minimum data element by comparing the updated j with the minimum index. Finally the exact minimum data element will be swapped with the first data element. Then the outer loop continues its iteration by incrementing i and the inner for loop starts from the new i+1; the algorithm continues like if there are two data elements to be compared as i and i+1. Both the outer and inner for loops use increment operator for iteration.

```
for (i=0; i<=n-2;i++)
  {
    min_index=i;
    for (j=(i+1); j<=n-1;j++)
        if (dataElement[j] < dataElement[min_index])
            min_index=j;
        swap (dataElement[i],dataElement[min_index]);
}
```

Example: write full program that can sort (5, 2, 3, 8, 1) using selection sort algorithm

Solution:

```
#include<iostream.h>
void swap(int &x, int &y)          //& means by address
  {
    int temp;
```

```
        temp=x;
        x=y;
        y=temp;
   }
int main()
{
    int a[5]={ 5, 2, 3, 8, 1};     // transform the unsorted data into array of size 5
    for(int i=0;i<4;i++)
        {
          int minIndex=i;
          for(int j=i+1;j<=4;j++)
            {
              if(a[j]<a[minIndex])
                {
                 minIndex=j;
                }
            }
          swap (a[i],a[minIndex]);
        }
   for(int k=0;k<5;k++)       //display the sorted data
      cout<<a[k];
      return 0;
}
/*output
1 2 3 5 8  */
```

The example below shows how a selection sort algorithm works

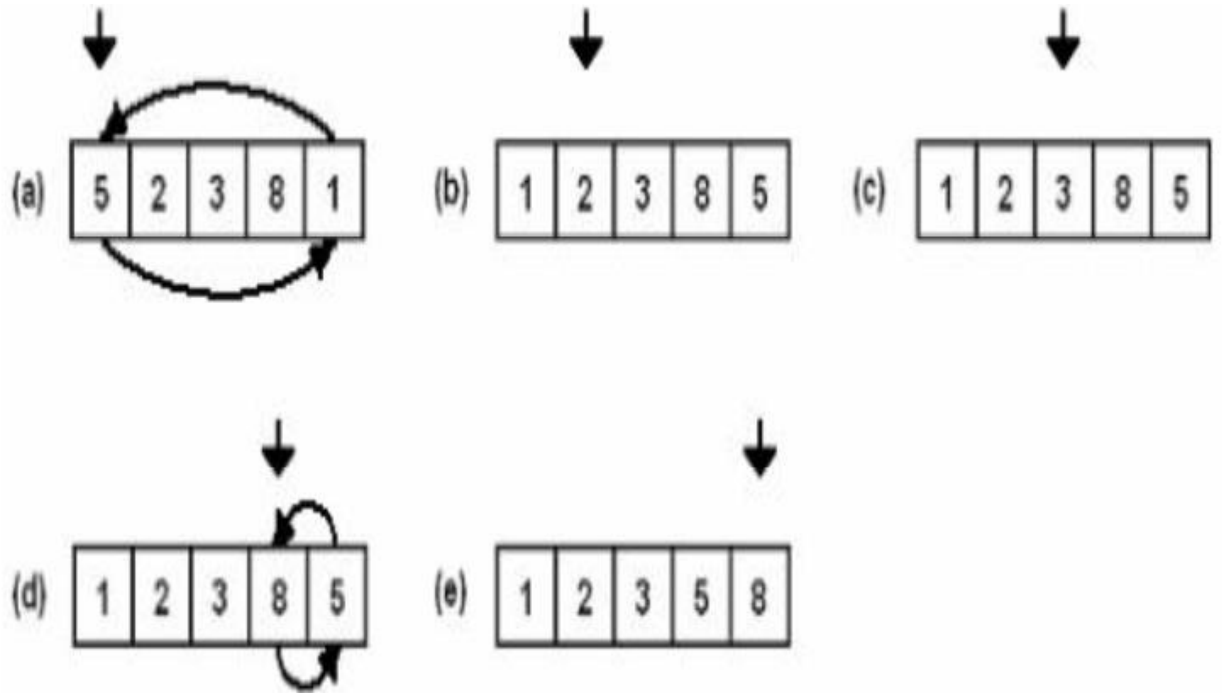


Fig. 2.1 Selection sort

Big-O analysis of Selection Sort Algorithm

Analysis involves number of comparisons and number of swaps. In the first pass there are n-1 comparisons and n-1 swaps. In the second pass there are n-2 comparisons and n-2 swaps and so on. In the $(n-1)^{th}$ pass there is 1 comparison and 1 swap. Totally there are n(n-1)/2 comparisons and n(n-1)/2 swaps. Therefore Big-O of selection sort algorithm is $n(n-1)/2 + n(n-1)/2 = O(n^2)$.

Activity 2.3

- ✓ Explain how selection sort algorithm works?
- ✓ Explain the increment decrement operator usage in selection sort algorithm?

2.1.3. Bubble Sort

It a method which uses the interchanging of adjacent pair of elements in the array. After each pass through the data one element (the largest or smallest element) will be moved to its proper place.

Implementation

Swapping algorithm: The swapping algorithm used in bubble sort algorithm is the same as the swapping algorithm that we have seen in insertion sort algorithm.

Comparison algorithm: it used to compare the values of two data elements to decide swapping is required or not between them. The algorithm uses nested for loop to compare the last data element with the second last data element and swaps them if the condition to swap is fulfilled. Then the algorithm comes down to the second last data element and compare it with the third last data element and swaps them if the condition to swap is fulfilled. The algorithm continues this operation until the inner for loop index j is equal to the index i of the data element contained by the outer for loop. i.e when i=j we are left with one data element and no need of comparison operation. To satisfy this the outer for loop starts i from index 0 means from the first data element and works until there are two data elements remaining for comparison which is i<n-1. The inner for loop starts j from the last data element which is n-1 and works until it is equal to i. The outer for loop uses the increment operator and the inner for loop uses decrement operator for iteration.

```
for (int i = 0; i < n-1; i++)
      for (int j = n-1; j > i; j--)
           if (data[j] < data[j-1])
              swap(data[j],data[j-1]);
```

Example: write full program that can sort (5, 2, 3, 8, 1) using bubble sort algorithm

Solution:

```
#include<iostream.h>
void swap(int &x, int &y)        //& means by address
    {
      int temp;
      temp=x;
```

```
    x=y;
    y=temp;
  }
int main()
{
   int a[5]={ 5, 2, 3, 8, 1};    //transform the unsorted data into array of size 5
   for(int i=0;i<4;i++)
     {
       for(int j=4;j>i;j--)
         {
           if(a[j]<a[j-1])
             {
               swap(a[j],a[j-1]);
             }
         }
     }
  for(int k=0;k<5;k++)        //display the sorted data
  cout<<a[k];
  return 0;
}
/*output
1 2 3 5 8
*/
```

The example below shows how a bubble sort algorithm works

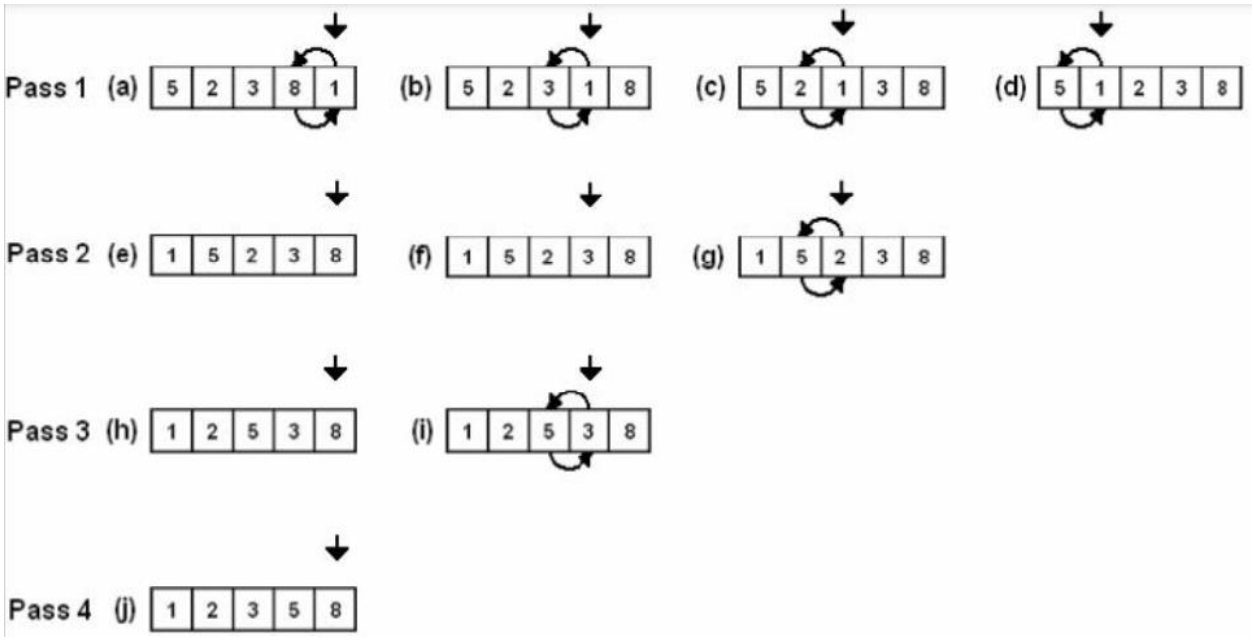


Fig. 2.2 Bubble sort

Big-O analysis of Bubble Sort Algorithm

Analysis involves number of comparisons and number of swaps. In the first pass there are n-1 comparisons and n-1 swaps. In the second pass there are n-2 comparisons and n-2 swaps and so on. In the $(n-1)^{th}$ pass there is 1 comparison and 1 swap. Totally there are n(n-1)/2 comparisons and n(n-1)/2 swaps. Therefore Big-O of bubble sort algorithm is $n(n-1)/2 + n(n-1)/2 = O(n^2)$.

Activity 2.4

- ✓ Explain how bubble sort algorithm works?
- ✓ Explain the increment decrement operator usage in bubble sort algorithm?

2.2. Searching Algorithm

Not even a single day pass, when we do not have to search for something in our day to day life, car keys, books, pen, mobile charger and what not. Same is the life of a computer, there is so much data stored in it, that whenever a user asks for some data, computer has to search it's memory to look for the data and make it available to the user. Searching is a process of finding an element from a collection of elements. To search an element in a given array, there are two popular algorithms available:

- Linear (Sequential) search
- Binary search

2.2.1. Linear (Sequential) Search

Linear (Sequential) search is a very basic and simple search algorithm. In Sequential search, we search an element or value in a given array by traversing the array from the starting, till the desired element or value is found. It compares the element to be searched with all the elements present in the array and when the element is matched successfully, it returns true, else it return false. Linear Search is applied on unsorted or unordered lists, when there are fewer elements in a list. The following statements summarizes sequential search

- It is natural way of searching
- The algorithm does not assume that the data is sorted.
- Search list from the beginning until key is found or end of list reached.

Features of Linear Search Algorithm

- It is used for unsorted and unordered small list of elements.
- It has a time complexity of O(n), which means the time is linearly dependent on the number of elements, which is not bad, but not that good too.
- It has a very simple implementation.

Implementation

Following are the steps of implementation that we will be following:

- Traverse the array using for loop.
- In every iteration, compare the target value with the current value of the array.
- If the values match, return true and stop iteration.

- If the values do not match, move on to the next array element.
- If no match is found, return false.

```
int i;
bool found =false;
for(i=0;i<n;i++;)
    if (DataElement[i]==key)
       {
 found=true;
 break;
       }
    return (found);
```

Example: To search the number 5 in the array {5, 2, 3, 8, 1}, sequential search will go step by step in a sequential order starting from the first element in the given array. In this example the algorithms stops after the first iteration and it returns true which is the best case analysis. The following table contains the index and data elements of an array.

Index	0	1	2	3	4	5	6	7
Data	D	X	C	A	E	V	B	W

Let us search V and S using sequential searching.

Key=V: after six comparison the algorithm returns found=true

Key=S: after 8 comparison the algorithm returns found=false

Big-O of Sequential Searching

For searching algorithm the dominant factor is comparison of keys.

How many comparisons are made in sequential searching? The maximum number of comparison is n, i.e. O (n).

Activity 2.5

- ✓ What are simple searching algorithms?
- ✓ Explain how sequential searching algorithm works?

2.2.2. Binary Searching

- The algorithm assumes that the data is sorted.
- Divide the list into halves (1)
- See which half of the list the key belongs (2)
- Repeat 1 and 2 until only one element remains
- If the remaining element is equal to search key then it will return true otherwise key not found and returns false.

Features of Binary Search Algorithm

- Binary Search is applied on the sorted array or list of large size.
- It's time complexity of O(log n) makes it very fast as compared to sequential searching algorithm.
- The only limitation is that the array or list of elements must be sorted for the binary search algorithm to work on it.

Implementation

The algorithm initializes three index variables bottom, middle and top. Bottom is initialized from the first element which is index 0, top is initialized from the last element which is index n-1 and middle is calculated as bottom + top divided by two and the integer part of the result will be considered as its value since it is declared as integer.

- Traverse the array using while loop.
- In every iteration, if top is greater than bottom divide the list into halves and compare the target value with the middle value.
- If the middle value less than the key, bottom is equal to middle+1 otherwise top is equal to middle.
- When only one data element is left its index is top and if the value of top matches the search key, return true move on to the next array element.
- If the value of top do not matches the search key, return false.

```
int Top =n-1, Bottom=0;middle;
    bool found=false;
    while (Top>bottom)
        {
          Middle=(Top+Bottom)/2;
            if( dataElement[middle]<key)
              Bottom=middle+1;
            else
              Top=middle;
        }
   if (dataElement[top]==key)
        found =true;
        return found;
```

Example: To search the number 5 in the array {1, 2, 3, 5, 8}, binary search first divide the list into halves and the middle data element will be 3. Now 3 will be compared with 5 and it is less than then bottom will be middle+1 which is 5. Now the half that may contain our search key is {5, 8}. Again divide it into halves and the middle data element will be 5. Now 5 will be compared with 5 and it is not less than then top will be middle which is 5. Now the value of bottom and top indexes is equal which means only one element is left and we cannot divide further. The value of top matches the search key which is 5. The algorithm returns true i.e. the search key is found.

Big-O of Binary Searching

How many comparisons are made in binary searching algorithm? The following table shows the number of comparisons made in binary searching algorithm.

N	Maximum number of comparison
1	1
2	2
4	3
8	4

16	5
32	16

Table 2.1 input size and number of comparisons in binary search

Then for n number of data elements, logn+1 number of comparison is needed; so, it is O(logn).

Activity 2.6

- ✓ Explain how binary searching algorithm works?
- ✓ Compare the efficiency of sequential and binary searching algorithms?

Chapter Two Review Questions

1. Write the pointer sort versions of the above sorting algorithms. Hint: use array of pointer instead of simple array to store the data elements and sort the data elements by pointer to pointer assignment.
2. Write a program that will implement all of the sorting algorithms. The program should accept different unsorted data elements from user and sort using all algorithms and should tell which algorithm is efficient for that particular unsorted data elements. Hint: use how many comparisons and swaps are done and the one with less number of swap and comparison number is considered as an efficient algorithm for that particular data elements.

Chapter Three: Linked Lists

3.1. Review on Pointer and Dynamic Memory allocation

Pointer

A pointer is a primitive data structure which holds the address of another variable.

Syntax:

```
data_type  * variable_name;   // declares a pointer to data_type
variable_name = & someother_variable;      // variable_name holds  someother_variable
address.
```

Example:

```
int number;
int  *ptr = &number;
```

The above code is equivalent to:

```
int number;
int *ptr;
ptr = &number;
```

Here the star is known as indirection or dereferencing operator which helps to access the contents of the variable pointed by the pointer. The ampersand symbol indicates the address of the variable that follows.

- & is the reference operator and can be read as "address of"
- * is the dereference operator and can be read as "value pointed by"

Allocating memory

There are two ways that memory gets allocated for data storage:

Compile Time (or static) Allocation

- Memory for named variables is allocated by the compiler
- Exact size and type of storage must be known at compile time
- For standard array declarations, this is why the size has to be constant

Dynamic Memory Allocation

- Memory allocated "on the fly" during run time dynamically allocated space usually placed in a program segment known as the heap or the free store

- Exact amount of space or number of items does not have to be known by the compiler in advance.
- For dynamic memory allocation, pointers are crucial

Dynamic Memory allocation

We can dynamically allocate storage space while the program is running, but we cannot create new variable names "on the fly" For this reason, dynamic allocation requires two steps:

- Creating the dynamic space.
- Storing its address in a pointer (so that the space can be accesed)

To dynamically allocate memory in C++, we use the new operator.

Allocating space with the new keyword

```
int * p;        // declare a pointer p
p = new int;    // dynamically allocate an int and load address into p
double * d;     // declare a pointer d
d = new double; // dynamically allocate a double and load address into d
// we can also do these in single line statements
int x = 40;
int * list = new int[x];
```

Accessing dynamically created space

So once the space has been dynamically allocated, how do we use it?

For single items, we go through the pointer. Dereference the pointer to reach the dynamically created target:

```
int * p = new int;      // dynamic integer, pointed to by p
*p = 10;                // assigns 10 to the dynamic integer
cout << *p;             // prints 10
```

For dynamically created arrays, you can use either pointer-offset notation, or treat the pointer as the array name and use the standard bracket notation:

```
double * numList = new double[size];        // dynamic array
for (int i = 0; i < size; i++)
        numList[i] = 0;                     // initialize array elements to 0
```

```
numList[5] = 20;                    // bracket notation
*(numList + 7) = 15;                // pointer-offset notation
                                    //   means same as numList[7]
```

De-allocation:

De-allocation is the "clean-up" of space being used for variables or other data storage
Compile time variables are automatically de-allocated based on their known extent. It is the programmer's job to de-allocate dynamically created space. To de-allocate dynamic memory, we use the delete operator.

De-allocation of dynamic memory

To de-allocate memory that was created with new, we use the unary operator delete. The one operand should be a pointer that stores the address of the space to be de-allocated:

```
int * ptr = new int;          // dynamically created int
delete ptr;                   // deletes the space that ptr points to
```

Note that the pointer ptr still exists in this example. That's a named variable subject to scope and extent determined at compile time. It can be reused:

```
ptr = new int[10];            // point p to a brand new array
```

To de-allocate a dynamic array, use this form:

```
delete [] name_of_pointer;
```

Example:

```
int * list = new int[40];     // dynamic array
delete [] list;               // de-allocates the array
list = 0;                     // reset list to null pointer
```

After de-allocating space, it's always a good idea to reset the pointer to null unless you are pointing it at another valid target right away.

Activity 3.1

- ✓ What is pointer?
- ✓ What is dynamic memory allocation?

Linked lists

Like arrays, Linked List is a linear data structure. Unlike arrays, linked list elements are not stored at a contiguous location; the elements are linked using pointers. A linked list is a data structure which is built from structures and pointers. The structure holds the data for each node (the name, address, age or whatever for the items in the list). It forms a chain of "nodes" with pointers representing the links of the chain and holding the entire thing together. Each node has a link to the next node in the series. The last node has a link to the special value NULL, to show that it is the last link in the chain. There is also another special pointer, called Head or Start, which points to the first link in the chain so that we can keep track of it.

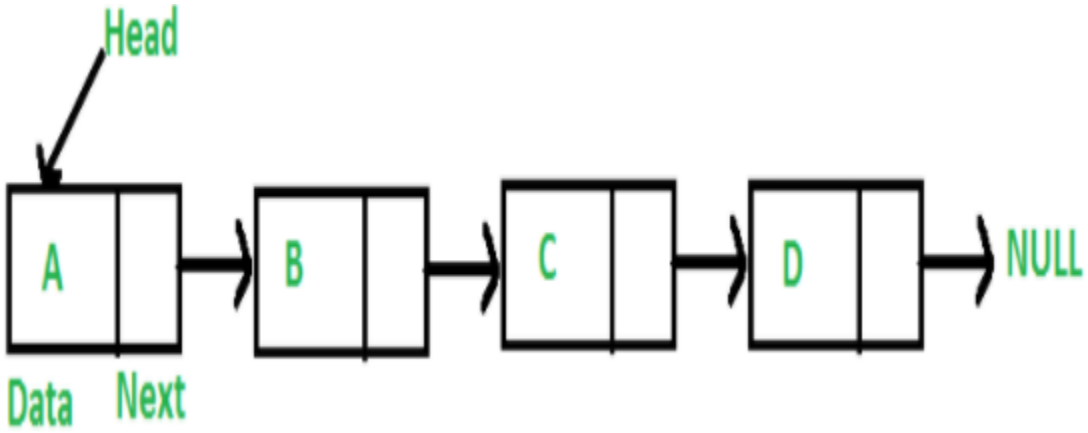


Fig. 3.1 Linked list

Why linked lists: Arrays can be used to store linear data of similar types, but arrays have the following limitations.

- The size of the arrays is fixed: So we must know the upper limit on the number of elements in advance. Also, generally, the allocated memory is equal to the upper limit irrespective of the usage.
- Inserting a new element in an array of elements is expensive because the room has to be created for the new elements and to create room existing elements have to be shifted but in Linked list if we have the head node then we can traverse to any node through it and insert new node at the required position. Deletion is also expensive with arrays until unless some special techniques are used and due to this so much work is being done which affects the efficiency of the code.

Advantages over array

- Dynamic size
- Ease of insertion/deletion

Drawbacks

- Random access is not allowed. We have to access elements sequentially starting from the first node (head node). So we cannot do binary search with linked lists efficiently with its default implementation.
- Extra memory space for a pointer is required with each element of the list.

Activity 3.2

- ✓ What is linked list?
- ✓ What is the Head or start pointer in linked list?
- ✓ Compare and contrast between array and linked list?

3.2. Singly Linked List and Its Implementation

A singly linked list is a type of linked list that is unidirectional, that is, it can be traversed in only one direction from head to the last node (tail). Each element in a linked list is called a node. A single node contains data and a pointer to the next node which helps in maintaining the structure of the list. The first node is called the head; it points to the first node of the list and helps us access every other element in the list. The last node, also sometimes called the tail, points to NULL which helps us in determining when the list ends.

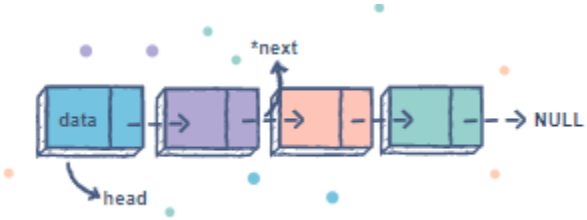


Fig. 3.2 Singly linked list

Common singly linked list operations

Add a node to the list

You can add a node at the front, the end or anywhere in the linked list.

Remove a node from the list

You can remove a node either from the front, the end or from anywhere in the list.

Traverse the list

You can traverse the data found in the list starting from head to the last node

Singly linked list Java implementation

When we implement linked list using Java programming the structure definition will be replaced by class definition. So, the first thing is defining the class for the linked list. Below is the class definition:

```
class Node {
   public int data;   //here we can have any member data
   public Node next;  //to make the chain in the list
   }
```

Now create a new class named SinglyLinkedList and create a Node type head to handle the start of the list and initialize it as null i.e. the list is empty. Note that head will permanently handle the start of the list.

```
public class SinglyLinkedList {
   private Node head=null;
}
```

The following Java code is used to add a node at the front of the list

```
public void addFront() {
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter data");
        int data = sc.nextInt();
        Node newNode = new Node();
        newNode.data = data;
        if(head==null){
        head=newNode;
        newNode.next=null;
        }
        else{
        newNode.next = head;
        head = newNode;
        }
   }
```

The following Java code is used to add a node at the end of the list

```
public void addEnd() {
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter data");
        int data = sc.nextInt();
        Node newNode = new Node();
        newNode.data = data;
        newNode.next = null;
        if(head==null){
           head=newNode;
        }
```

```
        else{
        Node last = head;
        while (last.next != null) {
            last = last.next;
        }
        last.next = newNode;
        }
    }
```

The following Java code is used to add a node at the middle of the list given position of the new node.

```
public void addMiddle() {
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter position");
        int pos = sc.nextInt();
        Node newNode = new Node();
        Node current,temp;
        if(pos<1){
                System.out.println("Enter proper position");
        }
        else if(head==null && pos>1){
                System.out.println("The position is not reachable");
        }
        else if(pos==1){
                addFront();
        }
        else if(head!=null){
                int j=1;
                Node last=head;
                while(last.next!=null){
                    last=last.next;
                    j++;
```

```
            }
            if(pos==j+1){
               addEnd();
            }
            else if(pos>j){
               System.out.println("The position is not reachable");
            }
            else{
               System.out.println("Enter data");
               int data = sc.nextInt();
               newNode.data = data;
               current = head;
               for(int i=1;i< pos-1;i++){
                  current = current.next;
               }
               temp=current.next;
               current.next = newNode;
               newNode.next=temp;
            }
      }
   }
```

The following Java code is used to remove a node from the front of the list

```
public void removeFront() {
      Node temp = head;
      if(temp==null){
         System.out.println("The list is empty");
      }
      else if(temp.next==null){
        head=null;
      }
      else{
```

```
            head = head.next;
        }
    }
```

The following Java code is used to remove a node from the end of the list

```
public void removeEnd() {
        Node temp = head;
        Node temp2=null;
        if(temp==null){
            System.out.println("The list is empty");
        }
        else if(temp.next==null){
          head=null;
        }
        else{
            while(temp.next!=null){
                temp2=temp;
                temp = temp.next;
            }
            temp2.next=null;
        }
    }
```

The following Java code is used to remove a node from the middle of the list given position of the node.

```
public void removeMiddle() {
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter position");
        int pos = sc.nextInt();
        Node current;
        if(head==null){
            System.out.println("The list is empty");
        }
```

```
    else if(head!=null){
        if(pos<1){
            System.out.println("Enter proper position");
        }
        else if(pos==1){
            removeFront();
        }
        else{
            int j=1;
            Node last=head;
            while(last.next!=null){
                last=last.next;
                j++;
            }
            if(pos==j){
                removeEnd();
            }
            else if(pos>j){
                System.out.println("The position is not reachable");
            }
            else{
                current = head;
                for(int i=1;i< pos-1;i++){
                    current = current.next;
                }
                current.next = current.next.next;
            }
        }
    }
}
```

The following Java code is used to display (traverse) the list starting from the front (head) to end (last).

```
public void displayList() {
      System.out.println("Displaying the List from head to last");
      Node current = head;
      if(current==null){
         System.out.println("The list is empty");
      }
      while (current != null) {
         System.out.println(current.data);
         current = current.next;
      }
   }
```

Activity 3.3

- ✓ Explain the properties of singly linked list?
- ✓ Write a Java code to sort singly linked list by selecting one of the sorting algorithms?

3.3. Doubly Linked List and Its Implementation

A doubly linked list is one where there are links from each node in both directions. It is a variation of linked list in which navigation is possible in both ways either forward or backward. Each link of a linked list contain a link to next link called Next and each link of a linked list contain a link to previous link called Prev. In a doubly linked list the first node prev is NULL and the last node next is NULL.

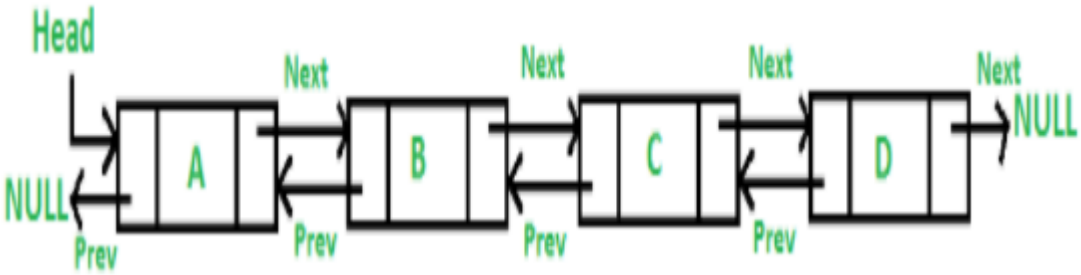


Fig. 3.3 Doubly linked list

Common doubly linked list operations

Add a node to the list

You can add a node at the front, the end or anywhere in the linked list.

Remove a node from the list

You can remove a node either from the front, the end or from anywhere in the list.

Traverse the list

You can traverse the data found in the list both forward and backward direction

Doubly linked list Java implementation

The first thing is defining the class for the linked list. Below is the class definition:

```
class Node {
   public int data;   //we can have any member data
   public Node prev;  //to make backward chain in the list
   public Node next;  //to make forward chain in the list
   }
```

Now create a new class named DoublyLinkedList and create a Node type head to handle the start of the list and initialize it as null i.e. the list is empty. Note that head will permanently handle the start of the list.

```
public class DoublyLinkedList {
   private Node head=null;
}
```

The following Java code is used to add a node at the front of the list

```
public void addFront() {
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter data");
        int data = sc.nextInt();
        Node newNode = new Node();
        newNode.data = data;
        newNode.prev=null;
        if(head==null){
        head=newNode;
        newNode.next=null;
        }
        else{
        newNode.next = head;
        head.prev=newNode;
        head = newNode;
        }
   }
```

The following Java code is used to add a node at the end of the list

```
public void addEnd() {
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter data");
        int data = sc.nextInt();
        Node newNode = new Node();
        newNode.data = data;
        newNode.next = null;
        if(head==null){
           newNode.prev=null;
```

```
        head=newNode;
    }
    else{
    Node last = head;
    while (last.next != null) {
        last = last.next;
    }
    last.next = newNode;
    newNode.prev=last;
    }
  }
```

The following Java code is used to add a node at the middle of the list given position of the new node.

```
public void addMiddle() {
    Scanner sc= new Scanner(System.in);
    System.out.println("Enter position");
    int pos = sc.nextInt();
    Node newNode = new Node();
    Node current,temp;
    if(pos<1){
        System.out.println("Enter proper position");
    }
    else if(head==null && pos>1){
        System.out.println("The position is not reachable");
    }
    else if(pos==1){
        addFront();
    }
    else if(head!=null){
        int j=1;
        Node last=head;
```

```
            while(last.next!=null){
                last=last.next;
                j++;
            }
            if(pos==j+1){
                addEnd();
            }
            else if(pos>j){
                System.out.println("The position is not reachable");
            }
            else{
                System.out.println("Enter data");
                int data = sc.nextInt();
                newNode.data = data;
                current = head;
                for(int i=1;i< pos-1;i++){
                    current = current.next;
                }
                temp=current.next;
                current.next = newNode;
                newNode.next=temp;
                newNode.prev=current;
                temp.prev=newNode;
            }
        }
    }
```

The following Java code is used to remove a node from the front of the list

```
public void removeFront() {
        Node temp = head;
        if(temp==null){
            System.out.println("The list is empty");
```

```
        }
        else if(temp.next==null){
          head=null;
        }
        else{
            head = head.next;
            head.prev=null;
        }
    }
```

The following Java code is used to remove a node from the end of the list

```
public void removeEnd() {
        Node temp = head;
        Node temp2=null;
        if(temp==null){
            System.out.println("The list is empty");
        }
        else if(temp.next==null){
          head=null;
        }
        else{
            while(temp.next!=null){
                temp2=temp;
                temp = temp.next;
            }
            temp2.next=null;
        }
    }
```

The following Java code is used to remove a node from the middle of the list given position of the node.

```
public void removeMiddle() {
        Scanner sc= new Scanner(System.in);
```

```
System.out.println("Enter position");
int pos = sc.nextInt();
Node current,temp;
if(head==null){
   System.out.println("The list is empty");
}
else if(head!=null){
   if(pos<1){
      System.out.println("Enter proper position");
   }
   else if(pos==1){
      removeFront();
   }
   else{
      int j=1;
      Node last=head;
      while(last.next!=null){
         last=last.next;
         j++;
      }
      if(pos==j){
         removeEnd();
      }
      else if(pos>j){
         System.out.println("The position is not reachable");
      }
      else{
         current = head;
         for(int i=1;i< pos-1;i++){
            current = current.next;
         }
```

```
                temp=current.next.next;
                current.next = temp;
                temp.prev=current;
            }
        }
    }
}
```

The following Java code is used to display (traverse) the list starting from the front (head) to end (last).

```
public void displayListForward() {
    System.out.println("Displaying the List from head to last");
    Node current = head;
    if(current==null){
        System.out.println("The list is empty");
    }
    while (current != null) {
        System.out.println(current.data);
        current = current.next;
    }
}
```

The following Java code is used to display (traverse) the list starting from the end (last) to front (head).

```
public void displayListBackward() {
    System.out.println("Displaying the List from last to head");
    Node current = head;
    Node last;
    if(current==null){
        System.out.println("The list is empty");
    }
    else{
        last=head;
```

```
        while(last.next!=null){
            last=last.next;
        }
        while (last != null) {
            System.out.println(last.data);
            last = last.prev;
        }
    }
}
```

Activity 3.4

- ✓ Explain the properties of doubly linked list?
- ✓ Write a Java code to sort doubly linked list by selecting one of the sorting algorithms?

3.4. Circular Linked Lists and Its Implementation

Circular linked list is a linked list where all nodes are connected to form a circle. In circular linked list the first and the last nodes are also connected to each other to form a circle. There is no NULL at the end. A circular linked list can be a singly circular linked list or doubly circular linked list.

Circular lists are useful in applications to repeatedly go around the list. For example, when multiple applications are running on a PC, it is common for the operating system to put the running applications on a list and then to cycle through them, giving each of them a slice of time to execute, and then making them wait while the CPU is given to another application. It is convenient for the operating system to use a circular list so that when it reaches the end of the list it can cycle around to the front of the list.

In a singly linked list, for accessing any node of the linked list, we start traversing from the first node. If we are at any node in the middle of the list, then it is not possible to access nodes that precede the given node. This problem can be solved by circular linked list in which any node can be a starting point. We can traverse the whole list by starting from any point. We just need to stop when the first visited node is visited again.

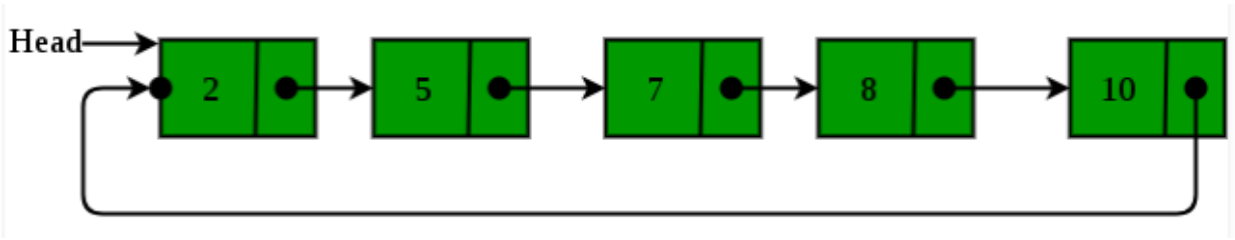


Fig. 3.4 Circular linked list

Circular Singly Linked List

Here, the address of the last node consists of the address of the first node.

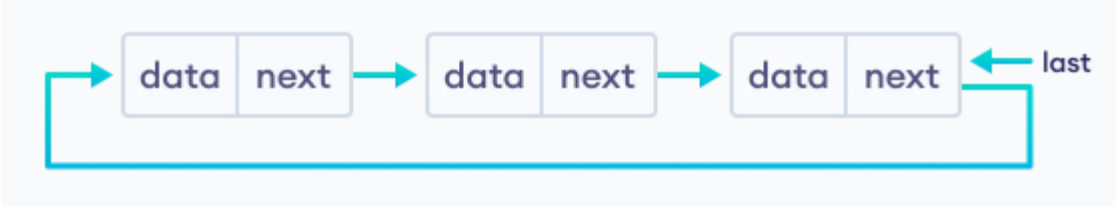


Fig. 3.5 Circular singly linked list

Circular Doubly Linked List

Here, in addition to the last node storing the address of the first node, the first node will also store the address of the last node.

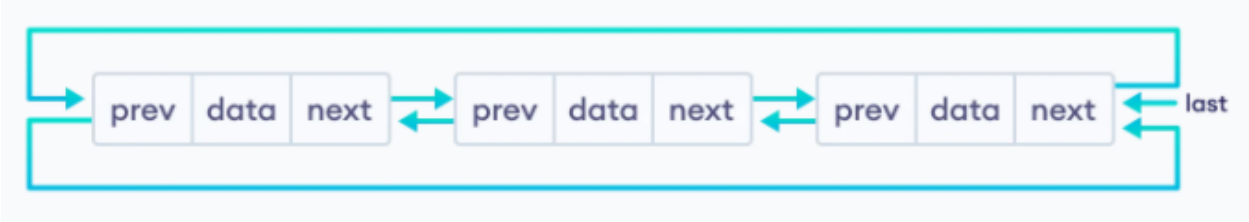
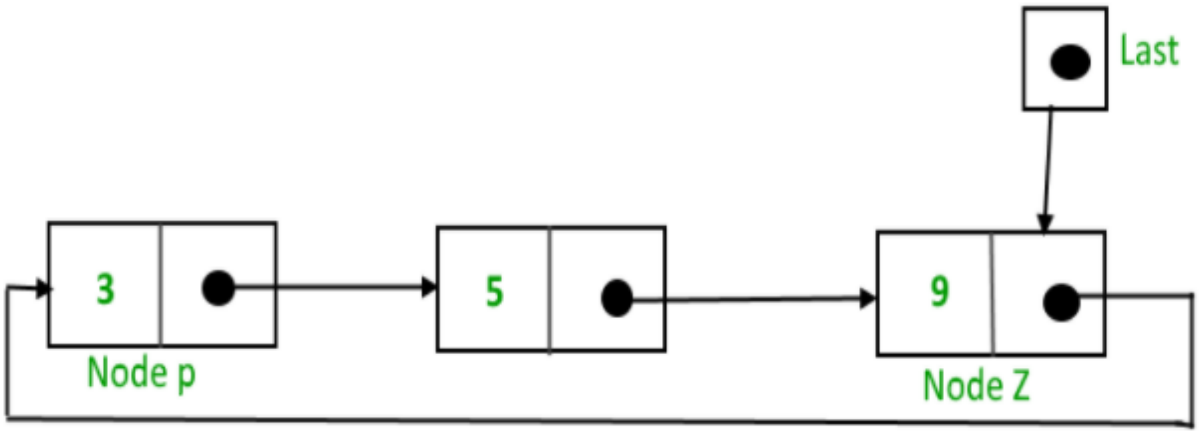


Fig. 3.6 Circular doubly linked list

Circular Singly Linked List Implementation

To implement a circular singly linked list, we take an external pointer that points to the last node of the list. If we have a pointer last pointing to the last node, then last -> next will point to the first node.



The pointer last points to node Z and last -> next points to node P.

Insertion in an empty List: initially, when the list is empty, the last pointer will be null. After insertion, the new node is the last node, so the pointer last points to the new node. And new node is the first and the last node, so the new node points to itself.

```
Node temp = new Node();
temp.data = data;
last = temp;
temp.next = last;
```

The following Java code is used to insert a node at the beginning of the list.

```
public void addBeginning() {
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter data");
        int data = sc.nextInt();
        Node newNode = new Node();
        newNode.data = data;
        if(last==null){
            last = newNode;
            newNode.next = last;
        }
        else{
            newNode.next = last.next;
            last.next = newNode;
        }
    }
```

The following Java code is used to insert a node at the end of the list.

```
public void addEnd() {
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter data");
        int data = sc.nextInt();
        Node newNode = new Node();
        newNode.data = data;
        if(last==null){
```

```
            last = newNode;
            newNode.next = last;
        }
        else{
            newNode.next = last.next;
            last.next = newNode;
            last = newNode;
        }
    }
```

The following Java code is used to insert a node at the middle (after a node) of the list.

```
public void addMiddle() {
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter position");
        int pos = sc.nextInt();
        Node newNode = new Node();
        if(pos<1){
                System.out.println("Enter proper position");
        }
        else if(last==null && pos>1){
                System.out.println("The position is not reachable");
        }
        else if(pos==1){
                addBeginning();
        }
        else if(last!=null){
                int j=1;
                Node temp=last.next;
                while(temp!=last){
                    temp=temp.next;
                    j++;
                }
```

```
            if(pos==j+1){
                addEnd();
            }
            else if(pos>j){
                System.out.println("The position is not reachable");
            }
            else{
                Node current,temp2;
                System.out.println("Enter data");
                int data = sc.nextInt();
                newNode.data = data;
                current = last.next;
                for(int i=1;i< pos-1;i++){
                    current = current.next;
                }
                temp2=current.next;
                current.next = newNode;
                newNode.next=temp2;
            }
        }
    }
```

The following Java code is used to remove a node from the beginning of the list.

```
public void removeBeginning() {
        Node current;
        if(last==null){
            System.out.println("The list is empty");
        }
        else if(last.next==last){
            last=null;
        }
        else{
```

```
            current=last.next;
            Node temp=current.next;
            last.next=temp;
        }
    }
```

The following Java code is used to remove a node at the end of the list.

```
public void removeEnd() {
        Node current,prev;
        if(last==null){
            System.out.println("The list is empty");
        }
        else if(last.next==last){
            last=null;
        }
        else{
            current=last.next;
            prev=null;
            while(current!=last){
                prev=current;
                current=current.next;
            }
            prev.next=last.next;
            last=prev;
        }
    }
```

The following Java code is used to remove a node from the middle of the list.

```
public void removeMiddle() {
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter position");
        int pos = sc.nextInt();
        Node current;
```

```
if(last==null){
   System.out.println("The list is empty");
}
else if(last!=null){
   if(pos<1){
      System.out.println("Enter proper position");
   }
   else if(pos==1){
      removeBeginning();
   }
   else{
      int j=1;
      Node temp=last.next;
      while(temp!=last){
         temp=temp.next;
         j++;
      }
      if(pos==j){
         removeEnd();
      }
      else if(pos>j){
         System.out.println("The position is not reachable");
      }
      else{
         current = last.next;
         for(int i=1;i< pos-1;i++){
            current = current.next;
         }
         current.next = current.next.next;
      }
   }
```

```
    }
  }
```

The following Java code is used to display the list.

```
  public void displayList() {
    System.out.println("Displaying the List from first node to last");
    Node current;
    if(last==null){
      System.out.println("The list is empty");
    }
    else{
      current=last.next;
      do{
        System.out.println(current.data);
        current=current.next;
      }while(current != last.next);
    }
  }
```

Activity 3.5

- ✓ Explain the properties of Circular linked list?
- ✓ Write a Java code to sort circular singly linked list by selecting one of the sorting algorithms?

Chapter Three Review Questions

1. Why is dynamic memory allocation and de-allocation important?
2. Write a Java code to search an item from singly linked list using linear searching algorithm?
3. Write a Java code to search an item from doubly linked list using linear searching algorithm?
4. Write a Java code to count the number of nodes in singly linked list?
5. Write a Java code to implement circular doubly linked list?

Chapter Four: Stacks

4.1.Properties of Stack

Stack: it is a data structure that has access to its data only at the end or top of the list. It operates on LIFO (last in first out) basis. There are many real-life examples of a stack. Consider the following examples:

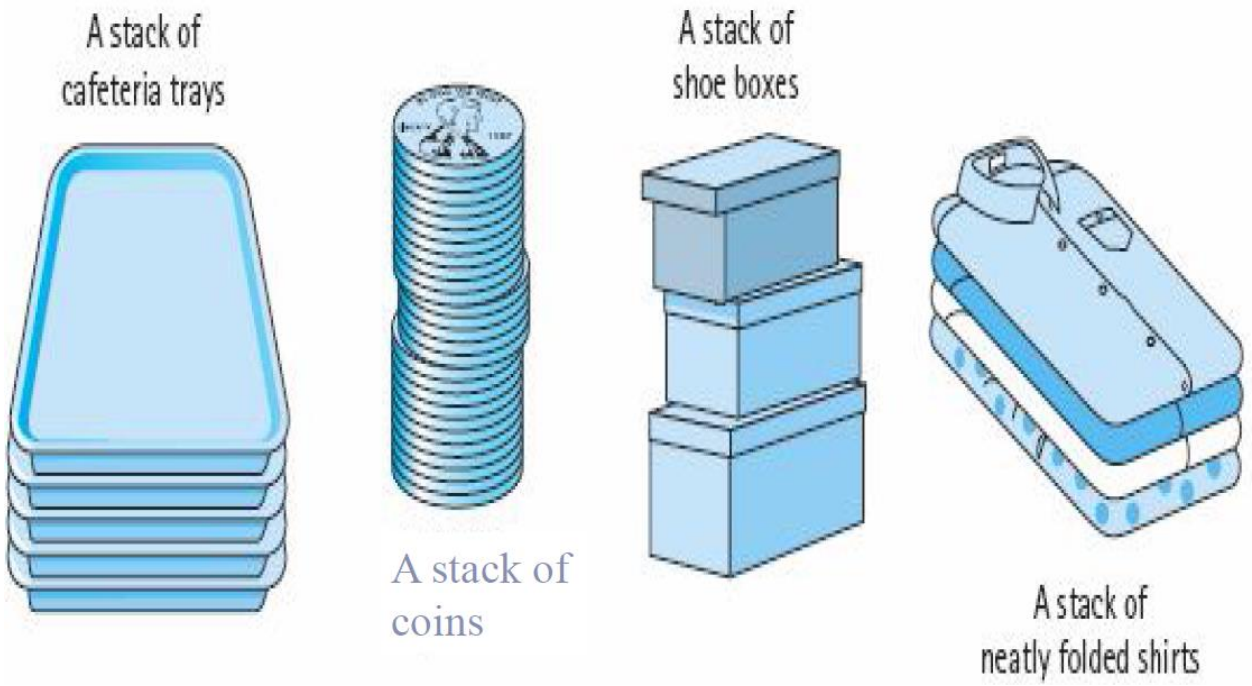


Fig. 4.1 Stack example

In programming terms, putting an item on top of the stack is called **push** and removing an item is called **pop**.

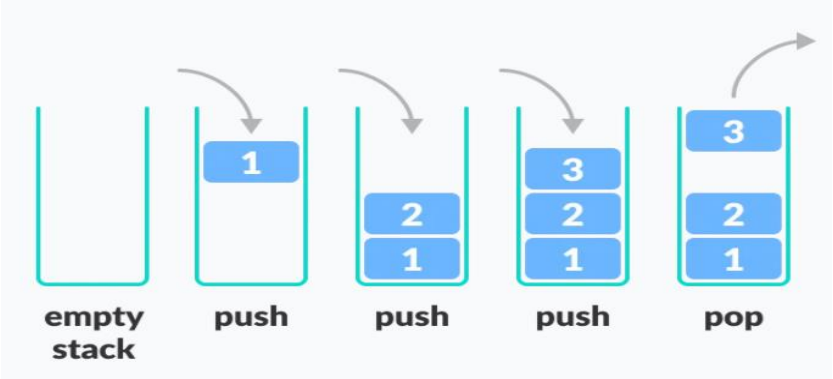


Fig. 4.2 Push and pop of stack

Stack uses a single pointer or index to keep track of the information or data on the stack.

There are some basic operations that allow us to perform different actions on a stack.

Push: Add an element to the top of a stack

Pop: Remove an element from the top of a stack

IsEmpty: Check if the stack is empty

IsFull: Check if the stack is full

The operations work as follows:

A pointer called TOP is used to keep track of the top element in the stack.

When initializing the stack, we set its value to -1 so that we can check if the stack is empty by comparing TOP == -1.

On pushing an element, we increase the value of TOP and place the new element in the position pointed to by TOP.

On popping an element, we return the element pointed to by TOP and reduce its value.

Before pushing, we check if the stack is already full

Before popping, we check if the stack is already empty

Activity 4.1

- ✓ Explain the properties of stack?
- ✓ Explain the basic operations of stack?

4.2.Array Implementation of Stack

Push an element to the stack

- Check if there is enough space in the stack
  - Yes :- increment top , store the element in num[top]
  - No space :- stack overflow

```
void push(int x)   {
        if(!isFull()) {
                top++;
                num[top]=x;
        }
        else
                cout<<"stack overflow";
}
```

Pop an element from the stack

- Check if there is data in the stack i.e. top>=0?
  - Yes: - copy/remove data at num[top], decrement top
  - No: - stack underflow

```
void pop() {
        int x;
        if(!isempty()) {
                x=num[top];
                cout<<"popped is: "<<x<<"\n";
                top --;
        }
        else
                cout<<"stack underflow";
}
```

Check the stack is empty

```
bool isempty() {
        return (top==-1)
}
```

Check the stack is full

```
bool isFull() {
        return(top==max_size – 1)
}
```

4.3.Linked list Implementation of Stack

The first thing is defining the class for the stack of linked list. Below is the class definition:

```
class Node {
   public int data;   //here we can have any member data
   public Node link;  //to make the stack in the list
   }
```

Now create a new class named StackofLinkedList and create a Node type top to handle the top of the list and initialize it as null i.e. the stack is empty. Note that top will permanently handle the top of the list.

```
public class StackofLinkedList {
   private Node top=null;
}
```

The following Java code is used to push data on to the stack of linked list

```
public void push(){
      Scanner sc= new Scanner(System.in);
      System.out.println("Enter data");
      int data = sc.nextInt();
      Node newNode = new Node();
      newNode.data = data;
      newNode.link=top;
      top=newNode;
```

```
}
```

The following Java code is used to pop data from the stack of linked list

```
public void pop(){
        if(top==null){
            System.out.println("stack underflow");
      }
      else{
            Node temp=top;
            top=top.link;
            System.out.println("Popped is: "+temp.data);
     }
}
```

The following Java code is used to display the data from the stack of linked list

```
 public void display(){
      System.out.println("Displaying from top to first");
      if(top==null){
         System.out.println("The stack is empty");
      }
      else{
         Node temp=top;
         while(temp!=null){
            System.out.println(temp.data);
            temp=temp.link;
         }
      }
   }
```

Activity 3.2

- ✓ Write a complete program that can implement stack of array?
- ✓ Write a complete program that can implement stack of linked list?

4.4.Applications of Stack Data Structure

Although stack is a simple data structure to implement, it is very powerful. The most common uses of a stack are:

To reverse a word: Put all the letters in a stack and pop them out. Because of the LIFO order of stack, you will get the letters in reverse order.

In browsers: The back button in a browser saves all the URLs you have visited previously in a stack. Each time you visit a new page, it is added on top of the stack. When you press the back button, the current URL is removed from the stack, and the previous URL is accessed.

Evaluation of Algebraic Expression: One of the compiler's tasks is to evaluate algebraic expression. Compiler must determine whether the right expression is a syntactically legal algebraic expression before evaluation can be done on the expression. The three algebraic expressions are: Infix, prefix and postfix.

The algebraic expression commonly used is infix. The term infix indicates that every binary operator appears between its operands. Example: A+B*C

To evaluate infix expression, the following rules were applied: Precedence rules, Association rules (associate from left to right) and Parentheses rules.Prefix and Postfix expressions are alternatives to infix expression. In prefix expression operator appears before its operand. Example: +AB In postfix expression operator appears after its operand. Example: AB+

Compilers use the stack to calculate the value of infix expressions by converting the expression to prefix or postfix form. The advantage of using prefix and postfix is that we don't need to use precedence rules, associative rules and parentheses when evaluating an expression.

Infix and Postfix (RPN) conversion: Reverse Polish Notation (RPN) is one where the operator follows its operands. Example: Infix notation A+B can be converted to postfix notation AB+ and infix notation (A+B)*C can be converted to postfix notation AB+C*

Function calls: The one that is found on the top of the stack is currently executed. When empty stack remain, the program will halt.

Chapter Four Review Questions

1. Explain the properties of stack with example?
2. Explain the applications of stack with example?
3. Write a program that can convert infix to postfix notation?
4. Write a program that computes a given postfix notation?

Chapter Five: Queue

5.1.Properties of Queue

A queue is a data structure which we can only add an item at the back or remove an item from the front. It operates on FIFO (first in first out) basis. The difference between stacks and queues is in removing. In a stack we remove the item the most recently added; in a queue, we remove the item the least recently added.

Example: waiting line (Line in a supermarket, Line at cafeteria, bank and so on…), Printing queue…

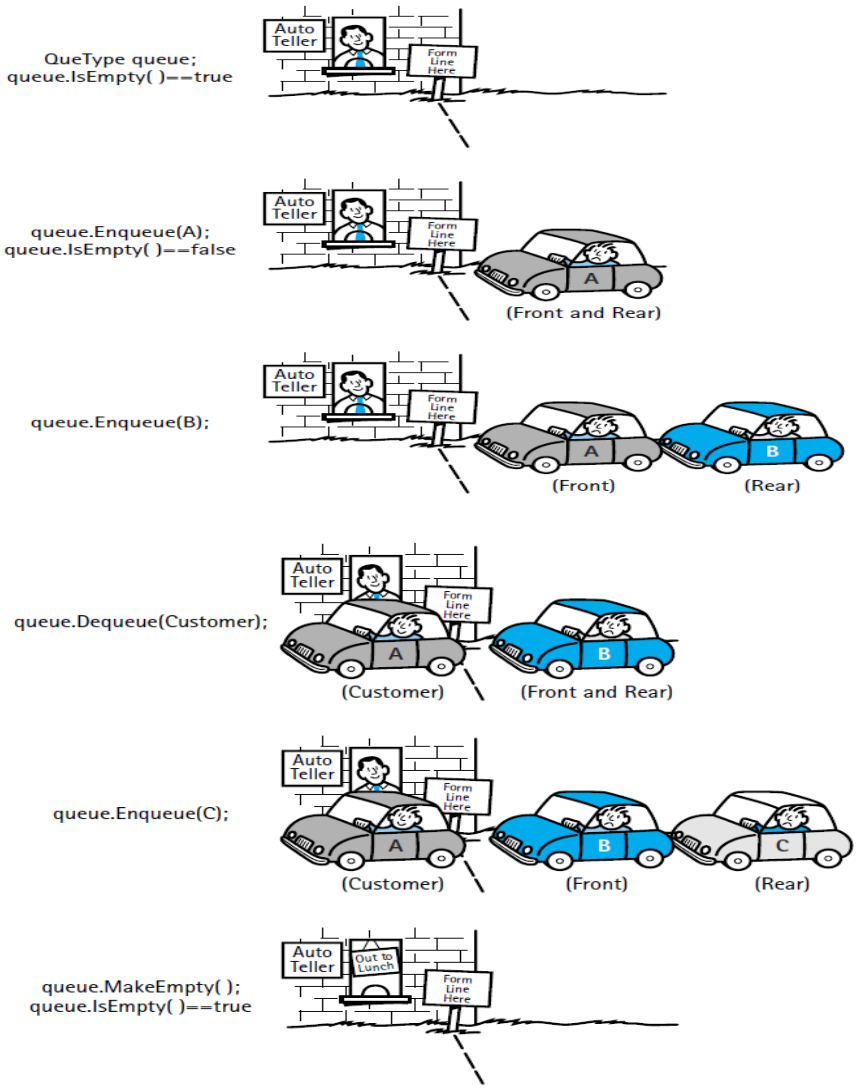


Fig. 5.1 Queue example

Basic Operations of Queue

A queue allows the following operations:

Enqueue: Add an element to the end of the queue

Dequeue: Remove an element from the front of the queue

IsEmpty: Check if the queue is empty

IsFull: Check if the queue is full

Queue operations work as follows:

It requires two pointers FRONT and REAR. FRONT track the first element of the queue and REAR track the last element of the queue. Initially, set value of FRONT and REAR to -1 and set queue size to 0

Enqueue Operation

Check if the queue is full

For the first element, set the value of FRONT to 0 and increase the REAR index by 1 then add the new element in the position pointed to by REAR. Increase queue size by 1

Dequeue Operation

Check if the queue is empty

Return the value pointed by FRONT and increase the FRONT index by 1.

Decrease queue size by 1

For the last element, reset the values of FRONT and REAR to -1.

Activity 5.1

- ✓ Explain the properties of queue?
- ✓ Explain the basic operations of queue?

5.2.Array Implementation of Queue

To enqueue data in to the queue

- Check if there is space
    - Yes:
        - queuesize==0?
            - Yes: increment front, rear and queuesize
            - No: Increment rear and queuesize
        - Store the data at rear
    - No: queue overflow

```
void enqueue(int x) {
        if(!isFull()) {
            if(qsize==0) {
                front++;
                qsize++;
                rear++;
        }
            else{
                rear++;
                qsize++;
            }
            num[rear]=x;
         }
         else
            cout<<"queue overflow";
}
```

To dequeue data from the queue

- Check if there is data (Queue size>0?)
    - Yes:
        - Remove the data from front
        - Increment front
        - Decrement queue size
    - No: queue underflow

```
void dequeue() {
    int x;
    if(!isempty()){
        x=num[front];
        front++;
        qsize--;
        cout<<"dequeued is: "<<x<<"\n";
    }
    else
        cout<<"queue underflow";
 }
```

Check the queue is empty

```
bool isempty() {
    return (qsize<=0);
}
```

Check the queue is full

```
bool isFull() {
    return(qsize>=max_size);
}
```

5.3.Linked list Implementation of Queue

The first thing is defining the class for the queue of linked list. Below is the class definition:

```
class Node {
   public int data;   //here we can have any member data
   public Node next;  //to make the queue in the list
   }
```

Now create a new class named QueueofLinkedList and create a Node type front and rear and initialize as null i.e. the queue is empty.

```
public class QueueofLinkedList {
   private Node front=null;
   private Node rear=null;
}
```

The following Java code is used to enqueue data in to the queue of linked list

```
public void enqueue(){
        Scanner sc= new Scanner(System.in);
      System.out.println("Enter data");
      int data = sc.nextInt();
      Node newNode = new Node();
      newNode.data = data;
        //newNode.next=null;
      if(front==null){
        front=rear=newNode;
        rear.next=null;
      }
      else{
        rear.next=newNode;
        rear=newNode;
        rear.next=null;
      }
    }
```

The following Java code is used to dequeue data from the queue of linked list

```
public void dequeue(){
        if(front==null){
        System.out.println("Queue underflow");
    }
    else{
          Node temp=front;
        front=front.next;
        System.out.println("Dequeued is: "+temp.data);
        }
  }
```

The following Java code is used to display the data from the queue of linked list

```
 public void display(){
    System.out.println("Displaying from front to rear");
    if(front==null){
        System.out.println("The queue is empty");
    }
    else{
        Node temp=front;
        while(temp!=null){
            System.out.println(temp.data);
            temp=temp.next;
        }
    }
  }
```

Activity 5.2

- ✓ Write a complete program that can implement queue of array?
- ✓ Write a complete program that can implement queue of linked list?

5.4.Double Ended Queue (Deque)

Deque or Double Ended Queue is a type of queue in which insertion and removal of elements can either be performed from the front or the rear. Thus, it does not follow FIFO rule (First In First Out).

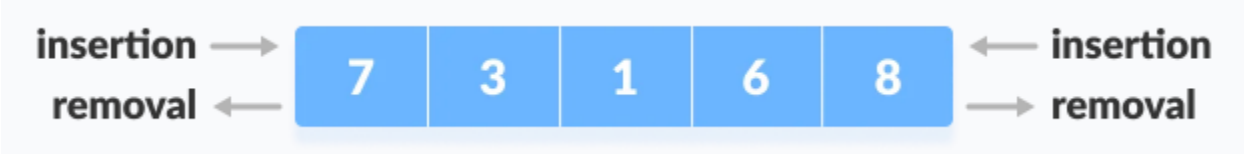


Fig. 5.2 Deque

Operations on Deque:

Mainly the following four basic operations are performed on Deque:

insertFront(): Adds an item at the front of Deque.

insertLast(): Adds an item at the rear of Deque.

deleteFront(): Deletes an item from front of Deque.

deleteLast(): Deletes an item from rear of Deque.

In addition to above operations, following operations are also supported

getFront(): Gets the front item from queue.

getRear(): Gets the last item from queue.

isEmpty(): Checks whether Deque is empty or not.

isFull(): Checks whether Deque is full or not.

Application of Deque

Since Deque supports both stack and queue operations, it can be used as both. The Deque data structure supports clockwise and anticlockwise rotations in O(1) time which can be useful in certain applications.

Deque Implementation

A Deque can be implemented either using a doubly linked list or circular array. Below is the circular array implementation of deque

Take an array (deque) of size n.

Set two pointers at the first position and set front = -1 and rear = 0.

Insert at the Front: This operation adds an element at the front.

Check the position of front.

If front < 1, reinitialize front = n-1 (last index).

Else, decrease front by 1.

Add the new key into array[front].

Insert at the Rear: This operation adds an element to the rear.

Check if the array is full.

If the deque is full, reinitialize rear = 0.

Else, increase rear by 1.

Add the new key into array[rear].

Delete from the Front: The operation deletes an element from the front.

Check if the deque is empty.

If the deque is empty (i.e. front = -1), deletion cannot be performed (underflow condition).

If the deque has only one element (i.e. front = rear), set front = -1 and rear = -1.

Else if front is at the end (i.e. front = n - 1), set go to the front front = 0.

Else, front = front + 1.

Delete from the Rear: This operation deletes an element from the rear.

Check if the deque is empty.

If the deque is empty (i.e. front = -1), deletion cannot be performed (underflow condition).

If the deque has only one element (i.e. front = rear), set front = -1 and rear = -1, else follow the steps below.

If rear is at the front (i.e. rear = 0), set go to the front rear = n - 1.

Else, rear = rear - 1.

Check Empty: This operation checks if the deque is empty. If front = -1, the deque is empty.

Check Full: This operation checks if the deque is full. If front = 0 and rear = n - 1 OR front = rear + 1, the deque is full.

5.5.Priority Queue

A priority queue is a **special type of queue** in which each element is associated with **priority value**. And, elements are served on the basis of their priority. That is, higher priority elements are served first. However, if elements with the same priority occur, they are served according

to their order in the queue. Generally, the value of the element itself is considered for assigning the priority. For example, the element with the highest value is considered the highest priority element. However, in other cases, we can assume the element with the lowest value as the highest priority element.

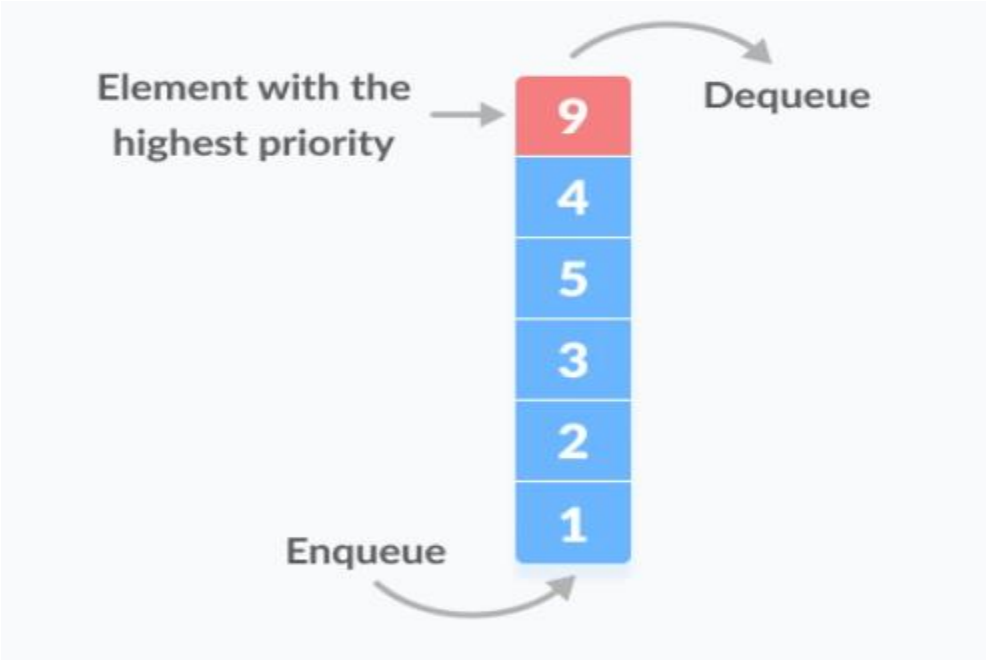


Fig. 5.3 Priority queue

In a queue, the first-in-first-out rule is implemented whereas, in a priority queue, the values are removed on the basis of priority. The element with the highest priority is removed first.

Implementation of Priority Queue

Priority queue can be implemented using an array, a linked list, a heap data structure, or a binary search tree. Among these data structures, heap data structure provides an efficient implementation of priority queues.

Priority Queue Operations

Inserting an Element into the Priority Queue

Inserting an element into a priority queue (max-heap) is done by the following steps.

Insert the new element at the end of the tree and then heapify the tree.

Deleting an Element from the Priority Queue

Deleting an element from a priority queue (max-heap) is done as follows:

Select the element to be deleted and swap it with the last element. Remove the last element and heapify the tree.

Peeking from the Priority Queue (Find max/min)

Peek operation returns the maximum element from Max Heap or minimum element from Min Heap without deleting the node. For both Max heap and Min Heap return rootNode.

5.6. Application of Queues

1. printserver

   Print(){

   Enqueue printqueue(document)

   }

   End_of_print(){

   Dequeue printqueue()

   }
2. Disk driver, the one first inserted will have first letter (eg. A:, B:, C:, D:, E…
3. Task scheduler in multiprocessing system
4. telephone calls in a busy environment
5. Simulation of waiting line: Line in a supermarket, line at cafeteria, bank and so on…

Activity 5.3

- ✓ Explain the properties of Double Ended Queue (Deque)?
- ✓ Explain the properties of Priority Queue?

Chapter Five Review Questions

1. What makes a queue different from stack?
2. What are the variations of queue?
3. Discuss the application of queue?
4. Write a Java program that can implement deque?
5. Write a Java program that can implement priority queue?

Chapter Six: Trees

Tree is an abstract model of a hierarchical non-linear data structure. A tree consists of a number of nodes connected by arcs. Unlike a real tree, tree data structures are typically depicted upside down, with the root at the top and the leaves at the bottom. Each node can have a number of child nodes, and the parent and child nodes are connected by arcs.

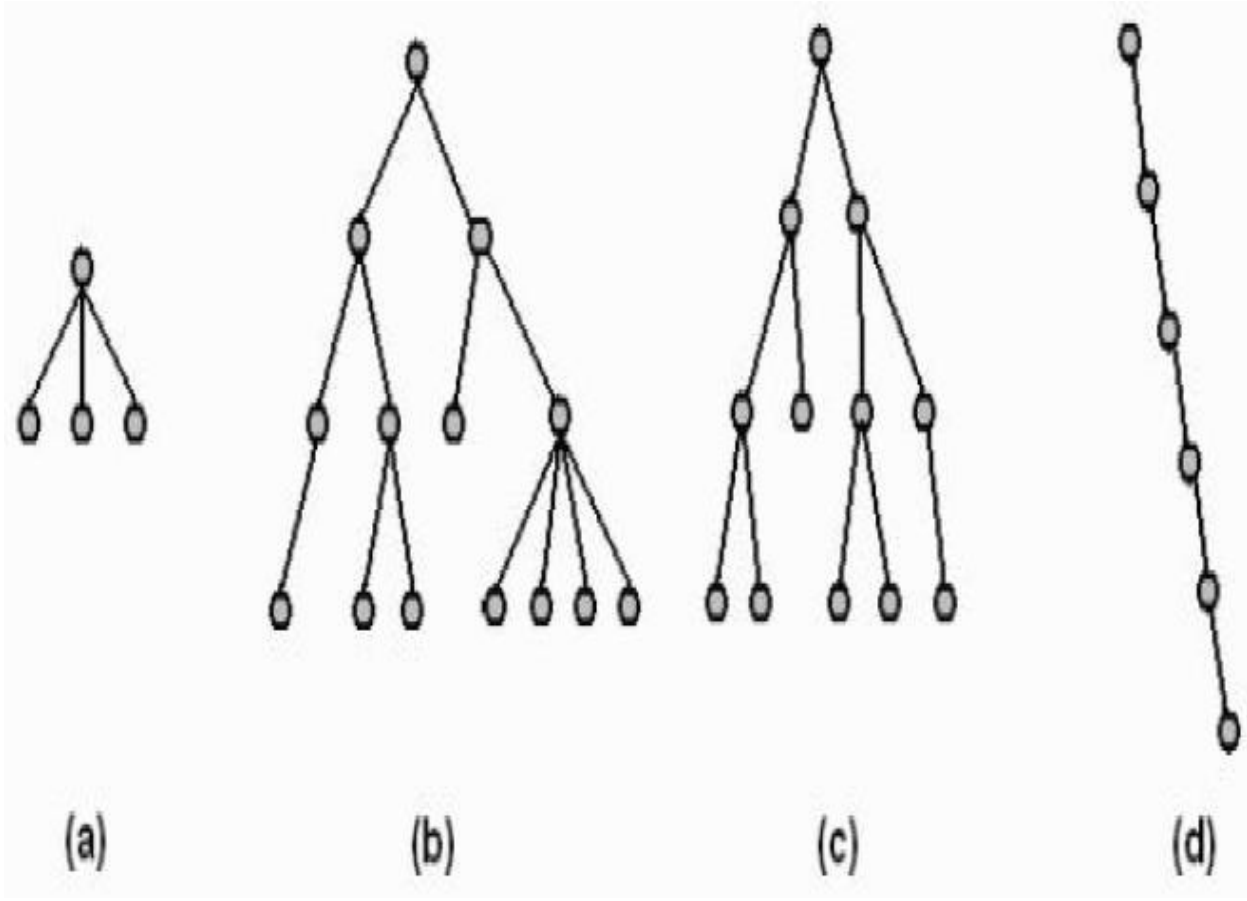


Fig. 6.1 Tree structure

The root is the top node that has no parent nodes. The leaves of the tree are those that have no child nodes. A path through the tree to a node is a sequence of arcs that connect the root to the node. The length of a path is the number of arcs in the path.

The level of a node is equal to the length of the path from the root to the node plus 1. The root node is said to be at level 1 of the tree, its children at level 2, and so on.

The height of a tree is equal to the maximum level of a node in the tree. For example, in the above figure (a) has a height of 2, (b) and (c) have heights of 4, and (d) has a height of 6.

In a tree data structure, the total number of edges from root node to a particular node is called as DEPTH of that Node. In a tree, the total number of edges from root node to a leaf node in the longest path is said to be Depth of the tree. In a tree, depth of the root node is 0.

In a tree data structure, the first node is called as Root Node. Every tree must have root node. We can say that root node is the origin of tree data structure. In any tree, there must be only one root node. We never have multiple root nodes in a tree.

In a tree data structure, the connecting link between any two nodes is called as EDGE. In a tree with 'N' number of nodes there will be a maximum of 'N-1' number of edges.

In a tree data structure, the node which is predecessor of any node is called as PARENT NODE. In simple words, the node which has branch from it to any other node is called as parent node.

In a tree data structure, the node which is descendant of any node is called as CHILD Node. In simple words, the node which has a link from its parent node is called as child node. In a tree, any parent node can have any number of child nodes. In a tree, all the nodes except root are child nodes.

In a tree data structure, nodes which belong to same Parent are called as SIBLINGS. In simple words, the nodes with same parent are called as Sibling nodes.

In a tree data structure, the node which does not have a child (or) node with degree zero is called as LEAF Node. In simple words, a leaf is a node with no child. The leaf nodes are also called as External Nodes.

In a tree data structure, the node which has at least one child is called as INTERNAL Node. In simple words, an internal node is a node with at least one child.
In a tree data structure, nodes other than leaf nodes are called as Internal Nodes. The root node is also said to be Internal Node if the tree has more than one node.

In a tree data structure, the total number of children of a node (or) number of subtrees of a node is called as DEGREE of that Node. In simple words, the Degree of a node is total number of children it has. The highest degree of a node among all the nodes in a tree is called as “Degree of Tree”. In a tree data structure, each child from a node forms a subtree recursively.

Trees can be useful data structures as they often express the structure of real world information more accurately. Example: organizational management structures, class hierarchy in Java, file system, etc.

Activity 6.1

- ✓ Explain what tree data structure is?
- ✓ List and explain the terminologies related to tree data structure?

6.1.Binary Tree and Binary Search Trees

A binary tree is a tree whose nodes have at most two children each: a left child and a right child. Nodes can have a single child (either left or right), or they can have no children, but they can never have more than two children.

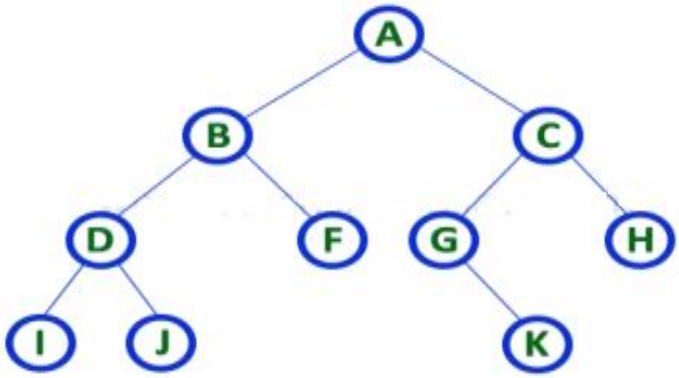


Fig. 6.2 Binary tree

Full binary tree: is a binary tree where each node has either zero or two children.

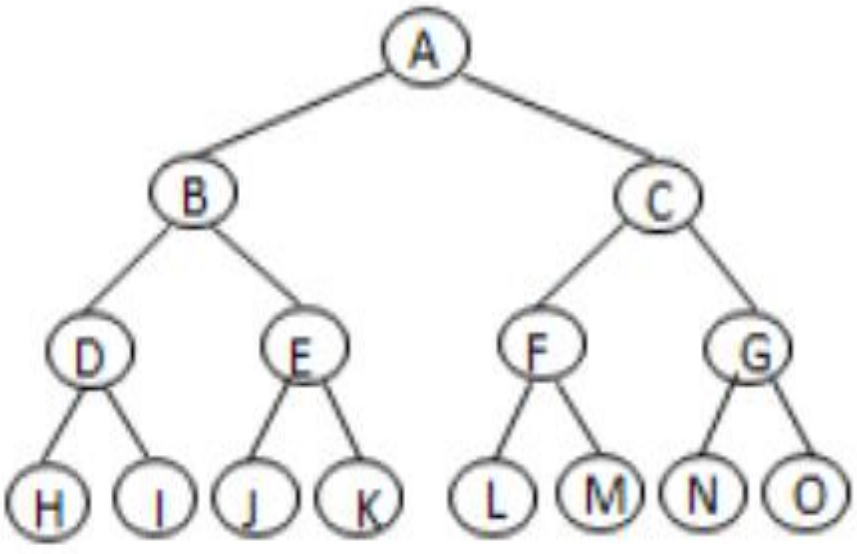


Fig. 6.3 Full binary tree

Complete Binary Tree: is a binary tree in which the level of the any leaf node is either H or H-1 where H is the height of the tree. The deepest level should also be filled from left to right.

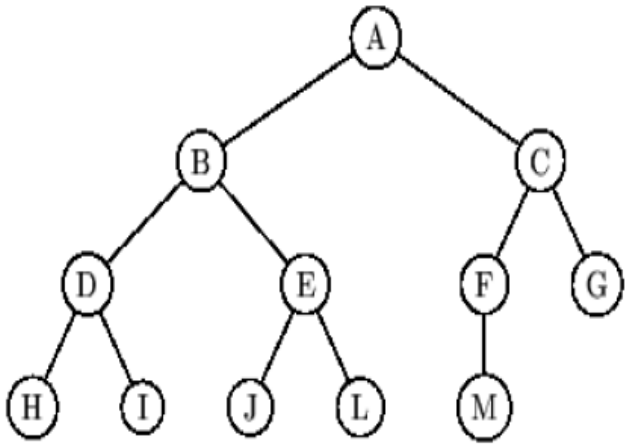


Fig. 6.4 Complete binary tree

Binary Search Tree (BST): It is a binary tree which can be empty or satisfies the following:

- Every node has a key and no two nodes have the same key
- The keys in the right sub tree are larger than the keys in the root
- The keys in the left sub tree are smaller than the keys in the root
- The left and right sub trees are also binary search trees.

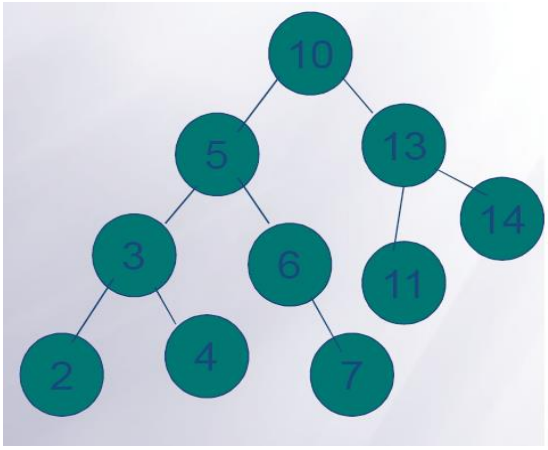


Fig. 6.5 Binary search tree

Activity 6.2

- ✓ Explain binary, full binary, complete, and binary search trees?
- ✓ Depict an example of binary search tree?

6.2.Basic Tree Operations

The first thing is defining the class for the tree. Below is the class definition:

```
class Node {
   public int num;
   public Node left;  //to manage the left subtree
   public Node right;  //to manage the right subtree
   }
```

Now create a new class named BinarySearchTree and create a Node type root and initialize as null i.e. the tree is empty.

```
public class BinarySearchTree {
        private Node root=null;
}
```

Insertion

To insert a node in to a binary search tree

If the tree is empty, the node to be inserted is made the root node. Otherwise, search the appropriate position and insert the node either in the left or right

```
public class BinarySearchTree {
   private static Node root=null;
   // used to insert a node in to BST
   public void insert() {
      String answer="";
      do{
         Scanner sc= new Scanner(System.in);
         System.out.println("Enter number");
         int num = sc.nextInt();
         Node newNode = new Node();
         newNode.num = num;
         int inserted=0;
         newNode.left=null;
         newNode.right=null;
```

```
    Node temp=root;
    if(temp==null){
        temp = newNode;
        root = newNode;
    }
    else{
        while(inserted==0){
            if(temp.num>newNode.num){
                if(temp.left==null){
                    temp.left=newNode;
                    inserted=1;
                }
                else{
                    temp=temp.left;
                }
            }
            else if(temp.num<newNode.num){
                if(temp.right==null){
                    temp.right=newNode;
                    inserted=1;
                }
                else{
                    temp=temp.right;
                }
            }
        }
    }
    Scanner sc2= new Scanner(System.in);
    System.out.println("Do you want to continue inserting yes/no?");
    answer = sc2.nextLine();
}while(answer.equals("yes"));
```

```
}
```

Searching

Compare the item that you are looking for with the root, and then push the comparison down into the left or right sub tree depending on the result of this comparison, until a match is found or a leaf is reached.

```
public void search(Node root, int num) {
    if(root==null){
        System.out.println("The key is not found");
    }
    else if (root.num==num){
        System.out.println("The key is found");
    }
    else if (root.num>num){
        return (search(root.left, num));
    }
    else{
        return (search(root.right, num));
    }
}
```

Counting nodes

Start from the root node and count all the nodes in the left and right sub trees

```
public int countNodes(Node root) {
    if(root==null){
        return 0;
    }
    else {
        int count = 1;
        count += countNodes(root.left);
        count += countNodes(root.right);
```

```
        return count;
    }
  }
```

Find height

Find the maximum of the sub trees and add 1

```
public int height(Node root) {
    if(root==null){
        return 0;
    }
    else {
        int leftHeight = height(root.left);
            int rightHeight= height(root.right);
        if (leftHeight > rightHeight){
            return(leftHeight+1);
        }
        else{
            return(rightHeight+1);
        }
    }
  }
```

6.3.Traversing in a binary tree

Tree structures can be traversed in many different ways.

Preorder traversal:

- Visit the root.
- Traverse the left subtree in preorder.
- Traverse the right subtree in preorder.

```
public void preorderTraversal(Node root) {
    if(root==null){
        return;
```

```
        }
        else {
            System.out.println(root.num);
            preorderTraversal(root.left);
            preorderTraversal(root.right);
        }
    }
```

In-order traversal:

- Traverse the left subtree in inorder.
- Visit the root.
- Traverse the right subtree in inorder.

```
public void inorderTraversal(Node root) {
        if(root==null){
            return;
        }
        else {
            inorderTraversal(root.left);
            System.out.println(root.num);
            inorderTraversal(root.right);
        }
    }
```

Post-order traversal: This is also called Depth-first traversal

- Traverse the left subtree in postorder.
- Traverse the right subtree in postorder.
- Visit the root.

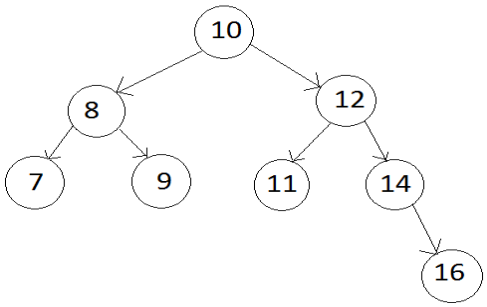
```
public void postorderTraversal(Node root) {
        if(root==null){
            return;
        }
        else {
            postorderTraversal(root.left);
```

```
        postorderTraversal(root.right);
        System.out.println(root.num);
    }
  }
```

Level-order traversal: visit every node on a level before going to a lower level. This is also called Breadth-first traversal

```
public void computeLevel(Node root) {
        int h = height(root);
        for (int i=1; i<=h; i++){
            levelorderTraversal(root, i);
        }
  }
public void levelorderTraversal(Node root, int level) {
    if(root==null){
        return;
    }
    else if(level==1) {
        System.out.println(root.num);
    }
    else if(level>1){
        levelorderTraversal(root.left,level-1);
        levelorderTraversal(root.right,level-1);
    }
  }
```

Example: traverse the following BST using all methods



Preorder traversal

10, 8, 7, 9, 12, 11, 14, 16

In-order traversal

7, 8, 9, 10, 11, 12, 14, 16

Post-order traversal

7, 9, 8, 11, 16, 14, 12, 10

Level order traversal

10, 8, 12, 7, 9, 11, 14, 16

Finding a node: find a node given its key (value)

```
Node findNode(int key){
      Node temp=root;
      while((temp!=null)&&(temp.num!=key)){
            if(key<temp.num){
            temp=temp.left;
         }
         else{
            temp=temp.right;
         }
      }
      return temp;
   }
```

Finding parent of a node: find parent of a node given the key of a node

```
Node findParentNode(int key){
      Node temp=root;
      Node temp2=null;
      while((temp!=null)&&(temp.num!=key)){
         temp2=temp;
         if(key<temp.num){
            temp=temp.left;
```

```
        }
        else{
            temp=temp.right;
        }
    }
    return temp2;
}
```

Find righty node: find the node with the largest value

```
Node findRightyNode(Node temp){
    Node righty=temp;
    while(righty.right!=null){
        righty=righty.right;
    }
    return righty;
}
```

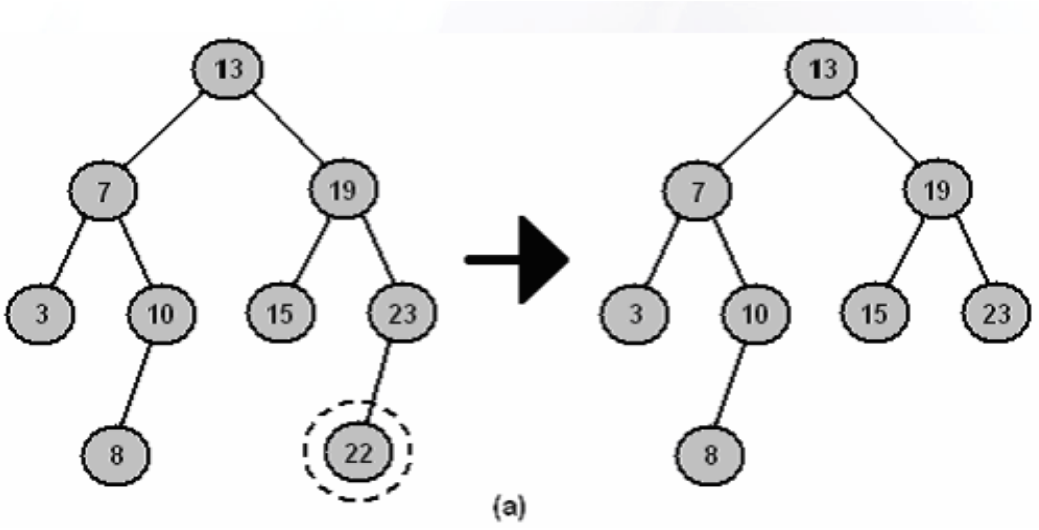
Find lefty node: find the node with the smallest value

```
Node findLeftyNode(Node temp){
    Node lefty=temp;
    while(lefty.left!=null){
        lefty=lefty.left;
    }
    return lefty;
}
```

Deletion in BST

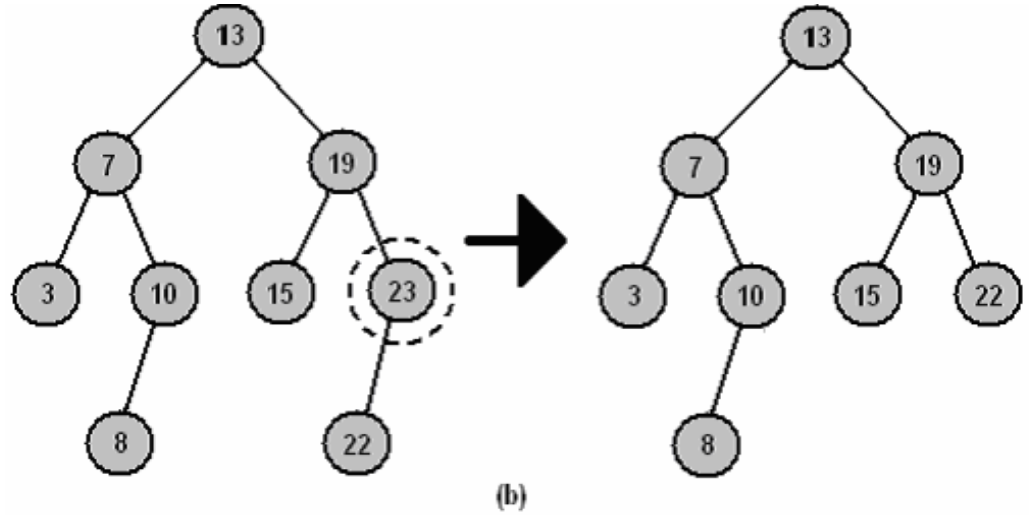
Deletion of a node from a binary search tree can be more problematic. The difficulty of the operation depends on the position of the node in the tree. There are three cases:

a) The node is a leaf. This is the easiest case to deal with. The appropriate pointer of its parent is set to null and the node deleted



(a)

b) The node has one child. This case is also not complicated. The appropriate pointer of its parent is modified to point to the child of the node



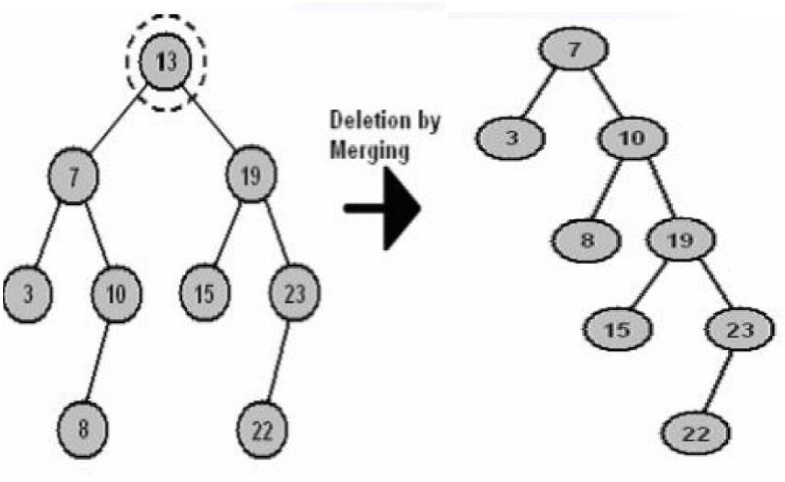
(b)

c) The node has two children. There is no simple way to delete nodes like this.

Deletion by Merging

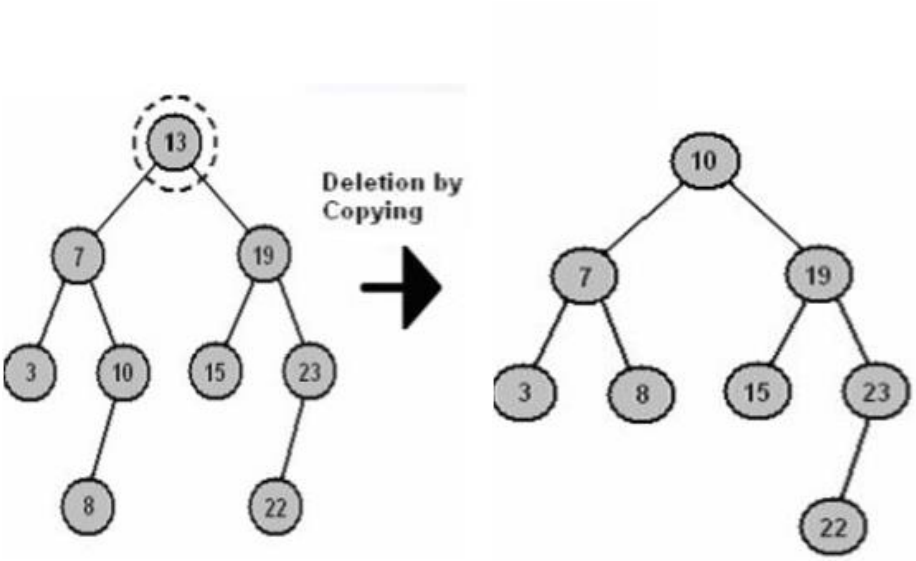
This algorithm works by merging the two sub trees of the node. In binary search trees, every value in the left sub tree is less than every value in the right sub tree, so if we can find the largest value in the left sub tree, then we can attach the right sub tree as the right child of this node. To find the largest value in the left sub tree we need only keep tracking the right child pointer until a

null is encountered. Symmetrically, this algorithm can also work by finding the smallest value in the right sub tree, and attaching the left sub tree to it.



Deletion by Copying

Deletion by copying works in a similar fashion to deletion by merging. First, we find the largest value in the left sub tree. Next, we copy this node, so that it replaces the node being deleted.
A slight complication with this algorithm can occur if the largest value in the left sub tree has a left child. In this case, the left child is attached to the nodes parent instead. For example, in Figure below the 10 node had a left child (node 8), so it was attached as the right child of 10's parent, namely 7.



Delete a node (Deletion by Copying)

```
public void deleteNode(int key) {
      Node temp=findNode(key);
      if(temp!=null) {
         //if both of child of node are null(leaf node)
         if(temp.left==null && temp.right==null){
            if(temp!=root){
               Node parent= findParentNode(key);
                     if(key<parent.num){
                           parent.left=null;
              }
              else{
                           parent.right=null;
                     }
             }
             else{
                root=null;
             }
         }
         //if only left child is not null
         else if(temp.left!=null && temp.right==null){
                  if(temp!=root){
                     Node parent=findParentNode(key);
               if(key<parent.num){
                     parent.left=temp.left;
               }
               else{
                     parent.right=temp.left;
               }
            }
            else{
```

```
            root=root.left;
        }
    }
    //if only right child is not null
    else if(temp.left==null && temp.right!=null){
            if(temp!=root){
                Node parent=findParentNode(key);
            if(key<parent.num){
                parent.left=temp.right;
            }
            else{
                parent.right=temp.right;
            }
        }
        else{
            root=root.right;
        }
    }
    //if both child are not null
    else{
             Node righty=findRightyNode(temp.left);
             Node parent=findParentNode(righty.num);
             temp.num=righty.num;
             if(parent!=temp){
             parent.right=righty.left;
            }
            else{
             temp.left=righty.left;
             }
     }
}
```

```
    else{
        System.out.println("The to be deleted is not found");
    }
}
```

Activity 6.3

- ✓ Write a Java program to delete a node using deletion by copy through lefty node?
- ✓ Write a Java program to delete a node using deletion by merging?

6.4.General Trees and Their Implementations

General trees are trees in which each node can have an arbitrary number of child nodes. Many organizations are hierarchical in nature, such as the military and most businesses. Consider a company with a president and some number of vice presidents who report to the president. Each vice president has some number of direct subordinates, and so on. If we wanted to model this company with a data structure, it would be natural to think of the president in the root node of a tree, the vice presidents at level 1, and their subordinates at lower levels in the tree as we go down the organizational hierarchy.

Because the number of vice presidents is likely to be more than two, this company's organization cannot easily be represented by a binary tree. We need instead to use a tree whose nodes have an arbitrary number of children. To distinguish them from binary trees, we use the term general tree.

Before discussing general tree implementations, we should first make precise what operations such implementations must support. Any implementation must be able to initialize a tree. Given a tree, we need access to the root of that tree. There must be some way to access the children of a node. In the case of the for binary tree nodes, this was done by providing member functions that give explicit access to the left and right child pointers. Unfortunately, because we do not know in advance how many children a given node will have in the general tree, we cannot give explicit functions to access each child. An alternative must be found that works for an unknown number of children.

One choice would be to provide a function that takes as its parameter the index for the desired child. That combined with a function that returns the number of children for a given node would support the ability to access any node or process all children of a node. Unfortunately, this view of access tends to bias the choice for node implementations in favor of an array-based approach, because these functions favor random access to a list of children. In practice, an implementation based on a linked list is often preferred.

An alternative is to provide access to the first (or leftmost) child of a node, and to provide access to the next (or right) sibling of a node.

General Tree Traversals

For general trees, preorder and postorder traversals are defined with meanings similar to their binary tree counterparts. Preorder traversal of a general tree first visits the root of the tree, then performs a preorder traversal of each subtree from left to right. A postorder traversal of a general tree performs a postorder traversal of the root's subtrees from left to right, then visits the root. Inorder traversal does not have a natural definition for the general tree, because there is no particular number of children for an internal node. An arbitrary definition: such as visit the leftmost subtree in inorder, then the root, then visit the remaining subtrees in inorder can be invented. However, inorder traversals are generally not useful with general trees.

Activity 6.4

- ✓ Explain the properties of general trees?
- ✓ Explain the difference between general trees and binary trees?

Chapter Six Review Questions

1. What makes tree different from linear data structures?
2. Discus the application of tree?
3. Discus the different tree traversal methods?
4. Discus the ways to delete a node in BST?
5. Write a Java program that can implement general tree?

Chapter Seven: Graph

7.1.Introduction

Trees by their nature have one limitation, which they can only represent relations of a hierarchical type, such as relations between parent and child. A generalization of tree, a graph, is a data structure in which this limitation is lifted.

A Graph is a non-linear data structure consisting of nodes and edges. The nodes are sometimes also referred to as vertices and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph can be defined as, A Graph consists of a finite set of vertices (or nodes) and set of Edges which connect a pair of nodes.

Graph is made up of a set of nodes called vertices and a set of lines called edges (or arcs) that connect the nodes. Edges describe relationships among the vertices. For instance, if the vertices are the names of the cities, the edges that link the vertices could represent roads between pairs of cities. Vertices represent whatever is the subject of our study like people, houses, cities, courses, and so on.

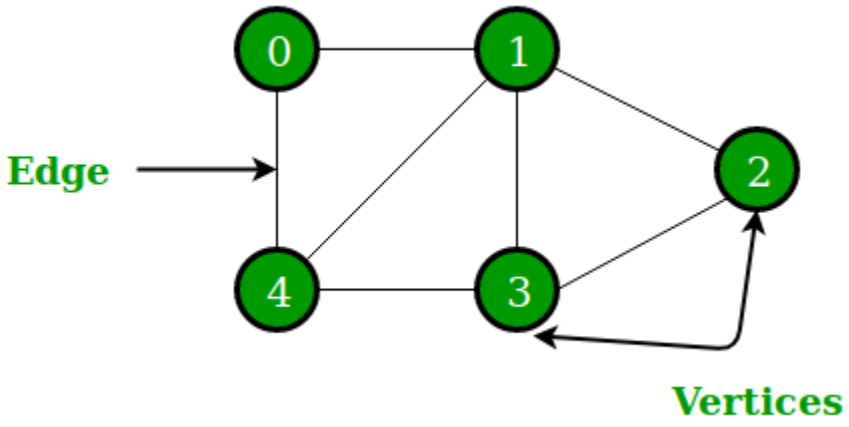


Fig. 7.1 Graph structure

Graphs are used to solve many real-life problems. Graphs are used to represent networks. The networks may include paths in a city or telephone network or circuit network. Graphs are also used in social networks like linkedIn, Facebook.

Let's try to understand this through an example. On facebook, everything is a node. That includes User, Photo, Album, Event, Group, Page, Comment, Story, Video, Link, Note...anything that has data is a node. Every relationship is an edge from one node to another. Whether you post a photo, join a group, like a page, etc., a new edge is created for that relationship.

All of facebook is then a collection of these nodes and edges. This is because facebook uses a graph data structure to store its data.

Activity 7.1

- ✓ Explain what graph data structure is?
- ✓ Explain the use of graph data structure?

7.2.Directed VS Undirected Graph

Undirected Graph

If the edges in the graph have no direction the structure is called undirected graph.

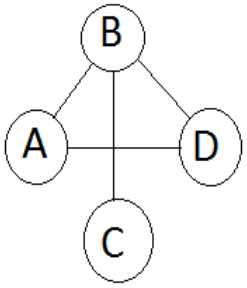
Directed graph

A graph whose edges are directed from one vertex to another is called a directed graph or digraph.

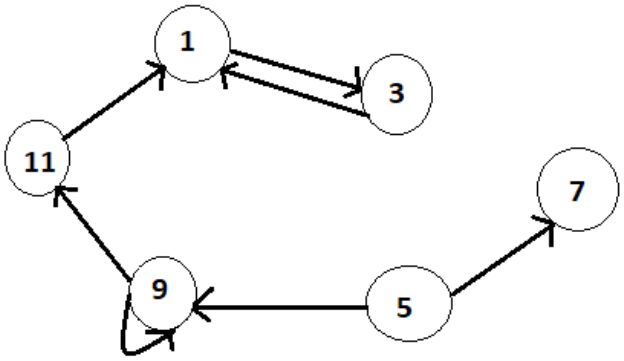
Formally a graph G is defined as G=(V,E)

- V(G) is a finite, non empty set of vertices.
- E(G) is a set of edges possibly empty set.

To specify the set of vertices, list them in set notation, within { } braces.



G1



G2

Graph 1: undirected graph V(G1)= {A,B,C,D} and E(G1)={(A,B),(A,D),(B,C),(B,D)}

Graph 2: directed graph V(G2)={1,3,5,7,9,11} and

E(G2)={(1,3),(3,1),(5,7),(5,9),(9,11),(9,9),(11,1)}

The set of edges is specified by listing a sequence of edges. To denote each edge, write the names of the two vertices it connects in parentheses, with a comma between them. If the graph is a digraph, the direction of the edge is indicated by which vertex is listed first. The edge (5,7) in graph 2 represents a link from vertex 5 to vertex 7. In digraphs, the arrows indicate the direction of the relationship.

If two vertices in a graph are connected by an edge, they are said to be adjacent. If the vertices are connected by a directed edge, then the first vertex is said to be adjacent to the second, and the

second vertex is said to be adjacent from the first. A tree is a special case of a directed graph, in which each vertex may be adjacent from only one other vertex (its parent node) and one vertex (the root) is not adjacent from any other vertex.

A path from one vertex to another consists of a sequence of vertices that connect them. For a path to exist, an uninterrupted sequence of edges must go from the first vertex, through any number of vertices, to the second vertex. In graph 2 a path goes from vertex 5 to vertex 3, but not the reverse.

Complete graph: a graph in which every vertex is directly connected to every other vertex. The number of edges in a complete directed graph with N vertices is N*(N-1). The number of edges in a complete undirected graph with N vertices is N*(N-1)/2.

Weighted Graph

Weighted graph: is a graph in which each edge carries a value. They used to represent applications in which the value of the connection between the vertices is important. For example: the distance between two cities as the edge value.

Graph Representation

Graphs are commonly represented in two ways:

1. Adjacency Matrix

An adjacency matrix is a 2D array of V x V vertices. Each row and column represents a vertex. Edge lookup (checking if an edge exists between vertex A and vertex B) is extremely fast in adjacency matrix representation but we have to reserve space for every possible link between all vertices (V x V), so it requires more space.

The basic operations like adding an edge, removing an edge, and checking whether there is an edge from vertex i to vertex j are extremely time efficient, constant time operations. If the graph is dense and the number of edges is large, an adjacency matrix should be the first choice.

By performing operations on the adjacent matrix, we can get important insights into the nature of the graph and the relationship between its vertices.

2. Adjacency List

An adjacency list represents a graph as an array of linked lists. The index of the array represents a vertex and each element in its linked list represents the other vertices that form an edge with the vertex. An adjacency list is efficient in terms of storage.

Activity 7.2

- ✓ Explain the types of graph?
- ✓ Explain the ways used to represent graph?

Common Operations on Weighted Graph

The first thing is defining the class for the graph. Below is the class definition:

```
class Node {
   public String city;    // as vertex
   public int distance;  // as weight of the edge
   Node[][] edges=new Node[10][10];  //to create edges between 10 vertices
   }
```

Now create a new class named WeightedGraph and create a Node type array of vertex for vertices.

```
public class WeightedGraph {
        Node[] vertex = new Node[10];
         int vIndex=0;
}
```

Insert vertex

```
public void insertVertex(){
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter the name of the city");
        vertex[vIndex].city = sc.nextLine();
        for(int i=0;i<vIndex;i++){
                vertex[vIndex].edges[vIndex][i]=null;
                vertex[i].edges[i][vIndex]=null;
        }
        vIndex++;
   }
```

Add weight of an edge

```
public void addEdge(){
        int weight,i=0;
        Node[] temp = new Node[10];
        int row=getFromIndex();
```

```
        int col=getToIndex();
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter the name of the city");
        temp[i].distance = sc.nextInt();
        vertex[row].edges[row][col]=temp[i];
    }
```

Return from index

```
public int getFromIndex(){
        int i=0;
        int j=0;
        Node[] temp=new Node[10];
        Scanner sc= new Scanner(System.in);
        System.out.println("Enter the name of from city");
        temp[j].city = sc.nextLine();
        while(!(temp[j].city.equals(vertex[i].city))){
        i++;
        }
        return i;
    }
```

Return to index

```
public int getToIndex(){
        int i=0;
        int j=0;
        Node[] temp=new Node[10];
        Scanner sc= new Scanner(System.in);
      System.out.println("Enter the name of to city");
      temp[j].city = sc.nextLine();
        while(!(temp[j].city.equals(vertex[i].city))){
        i++;
```

```
        }
        return i;
    }
```

Display weight of an edge

```
public void displayWeight(){
        Node[] temp=new Node[10];
        int i=0;
        int row,col;
        row=getFromIndex();
        col=getToIndex();
        System.out.println("The distance from "+vertex[row].city+" to "+vertex[col].city+" is  ");
        temp[i]=vertex[row].edges[row][col];
        if(temp[i]!=null)
                System.out.println(temp[i].distance+" KM");
        else
                System.out.println("Unknown!");
    }
```

Delete weight of an edge

```
public void deleteEdge(){
      int row,col;
      row=getFromIndex();
      col=getToIndex();
      vertex[row].edges[row][col]=null;
      System.out.println("The edge between "+vertex[row].city+"  and  "+vertex[col].city+"  is
      deleted");
    }
```

7.3.Traversing Graph

Breadth First Traversal or Breadth First Search is a recursive algorithm for searching all the vertices of a graph or tree data structure. Breadth-First Traversal for a graph is similar to Breadth-First Traversal of a tree. The only difference here is, unlike trees, graphs may contain cycles.

BFS algorithm

A standard BFS implementation puts each vertex of the graph into one of two categories:

1. Visited
2. Not Visited

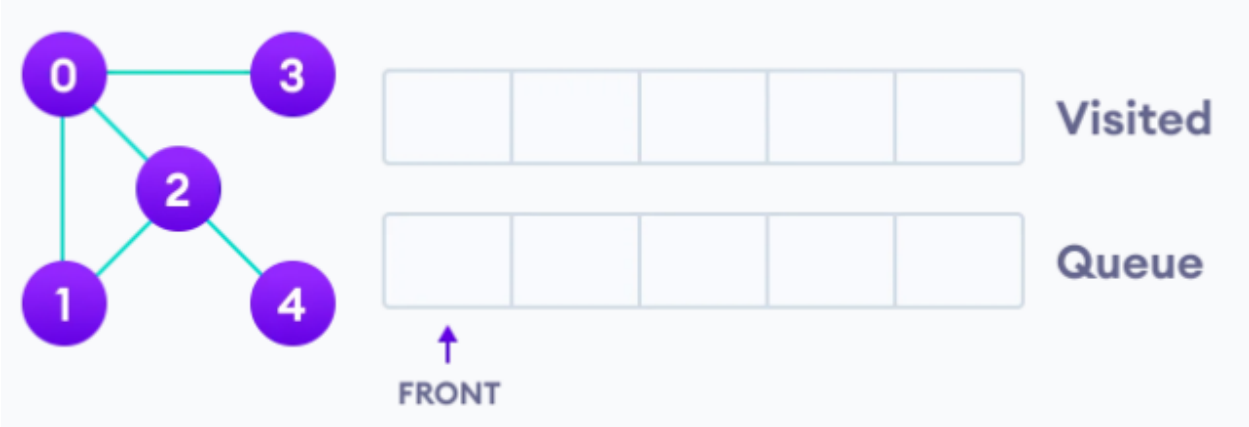
The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

The algorithm works as follows:

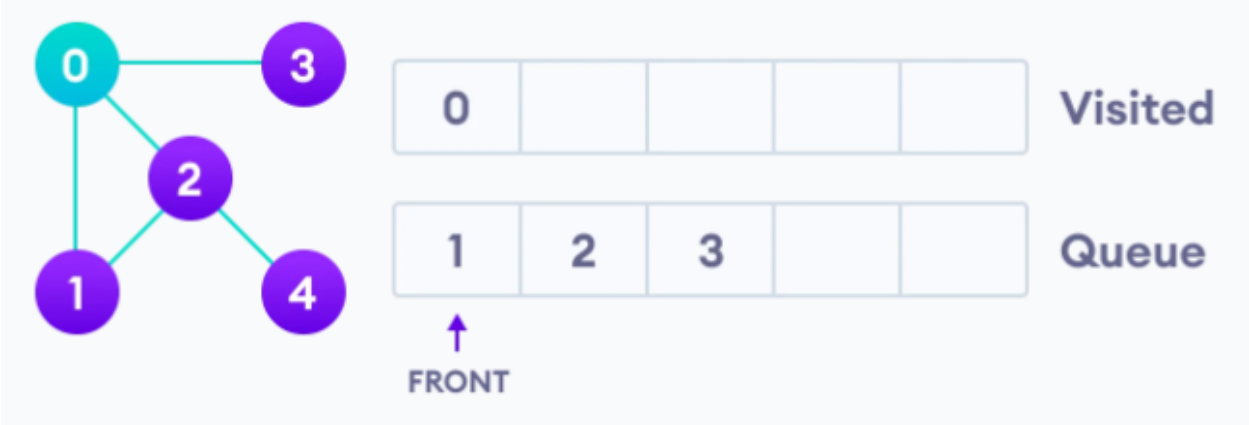
1. Start by putting any one of the graph's vertices at the back of a queue.
2. Take the front item of the queue and add it to the visited list.
3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the back of the queue.
4. Keep repeating steps 2 and 3 until the queue is empty.

The graph might have two different disconnected parts so to make sure that we cover every vertex, we can also run the BFS algorithm on every node.

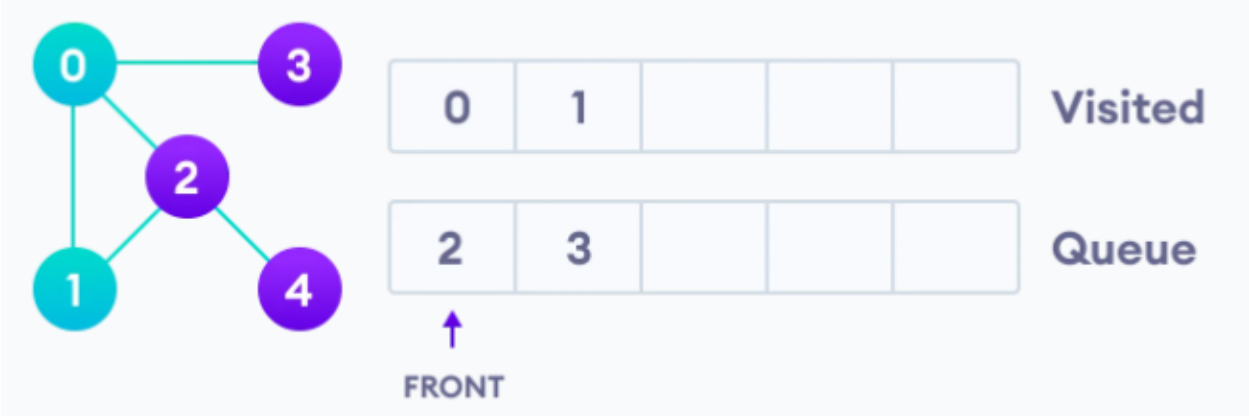
Let's see how the Breadth First Search algorithm works with an example. We use an undirected graph with 5 vertices.



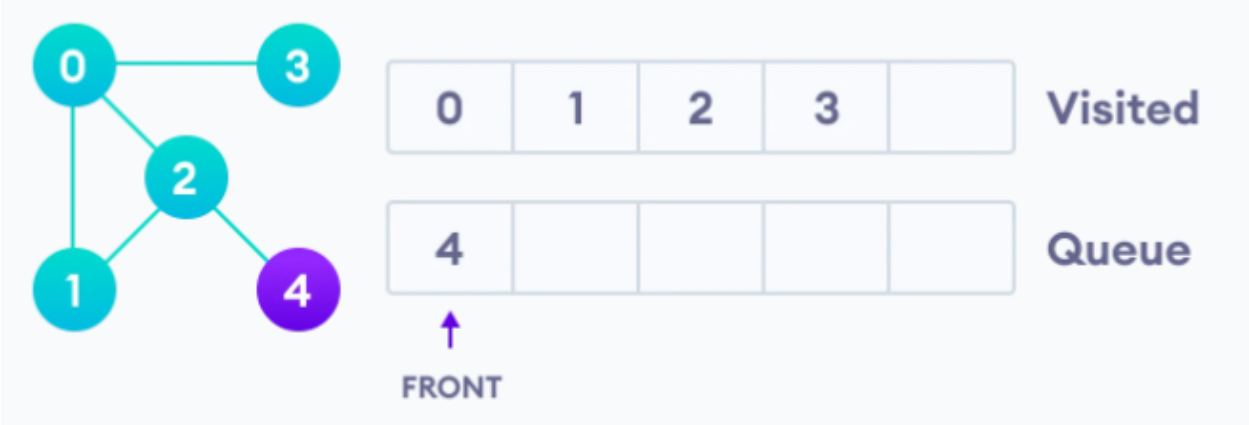
We start from vertex 0, the BFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the stack.



Next, we visit the element at the front of queue i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.



Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the back of the queue and visit 3, which is at the front of the queue.



Only 4 remains in the queue since the only adjacent node of 3 i.e. 0 is already visited. We visit it.

Since the queue is empty, we have completed the Breadth First Traversal of the graph.

Depth first Search or Depth first traversal is a recursive algorithm for searching all the vertices of a graph or tree data structure.

Depth First Search Algorithm

A standard DFS implementation puts each vertex of the graph into one of two categories:

1. Visited
2. Not Visited

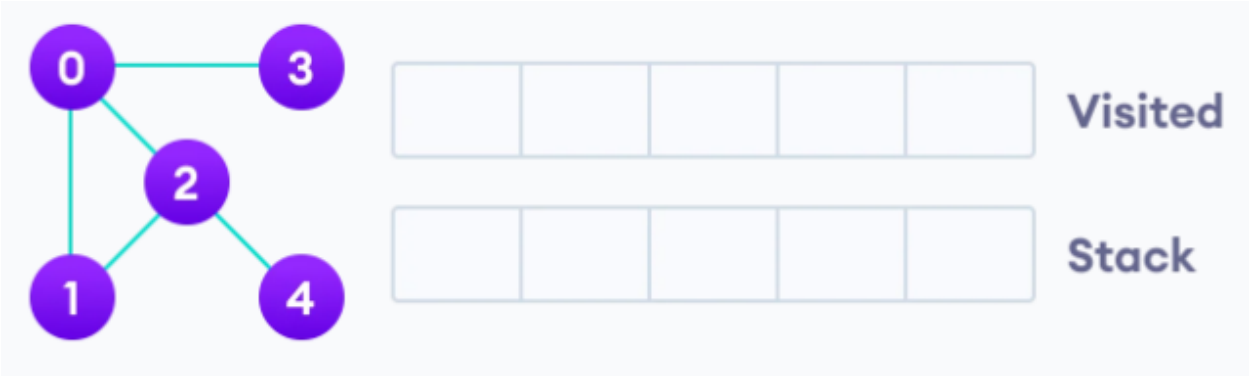
The purpose of the algorithm is to mark each vertex as visited while avoiding cycles.

The DFS algorithm works as follows:

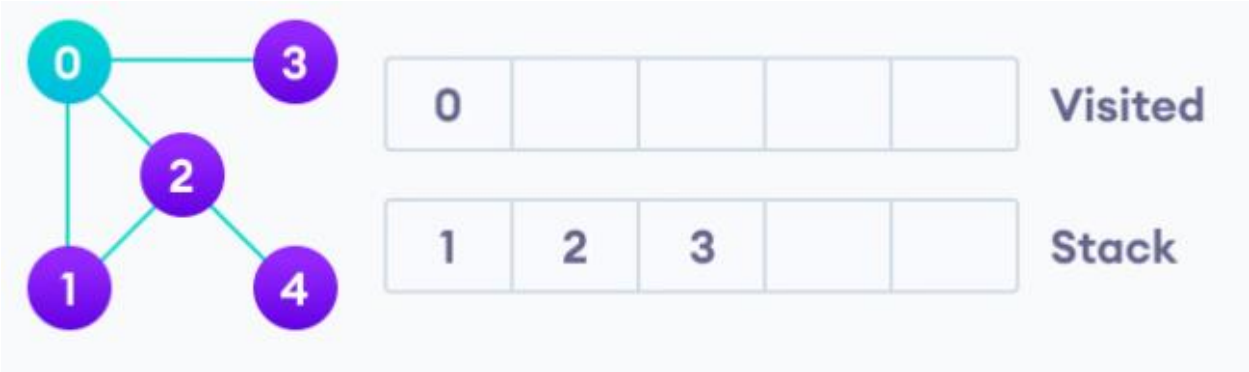
1. Start by putting any one of the graph's vertices on top of a stack.
2. Take the top item of the stack and add it to the visited list.

3. Create a list of that vertex's adjacent nodes. Add the ones which aren't in the visited list to the top of the stack.
4. Keep repeating steps 2 and 3 until the stack is empty.

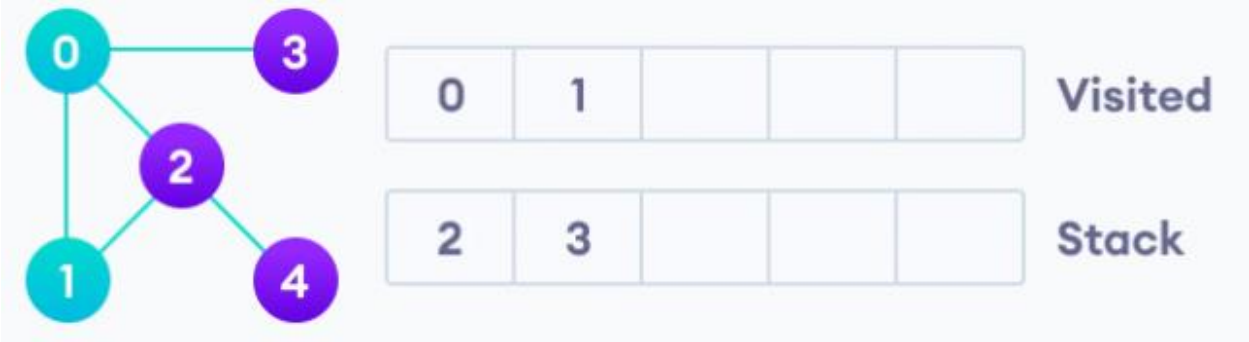
Let's see how the Depth First Search algorithm works with an example. We use an undirected graph with 5 vertices.



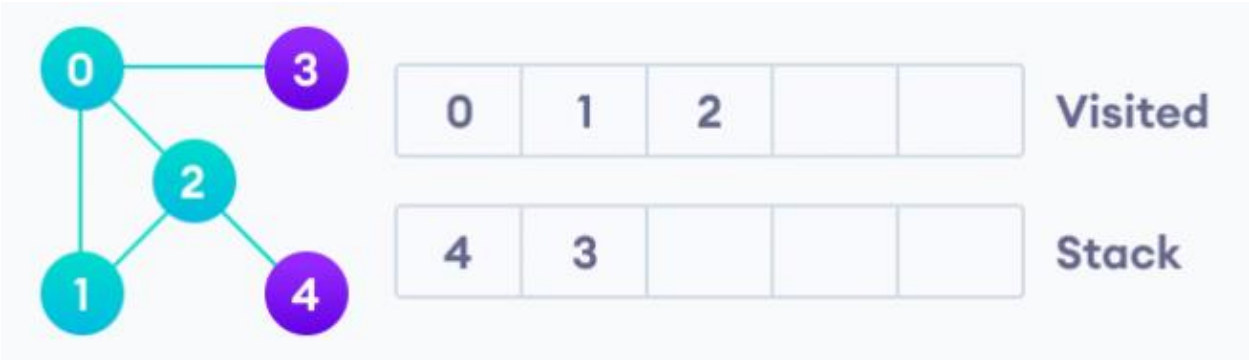
We start from vertex 0, the DFS algorithm starts by putting it in the Visited list and putting all its adjacent vertices in the stack.

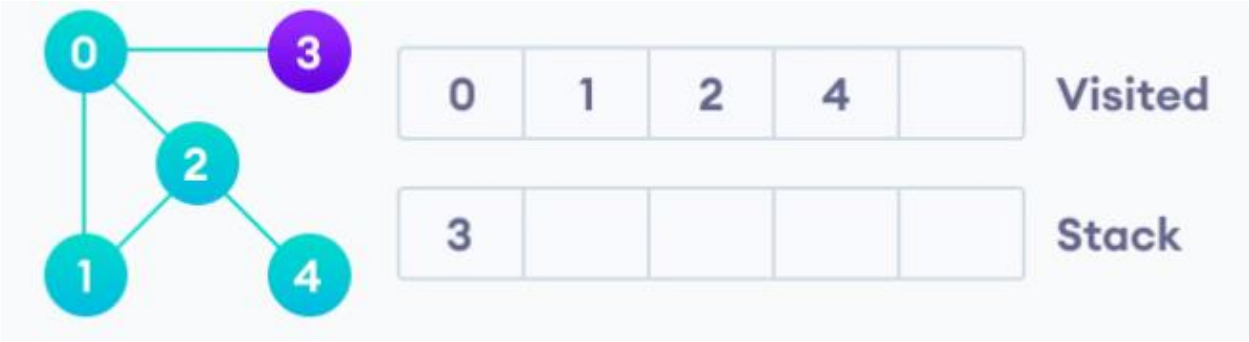


Next, we visit the element at the top of stack i.e. 1 and go to its adjacent nodes. Since 0 has already been visited, we visit 2 instead.

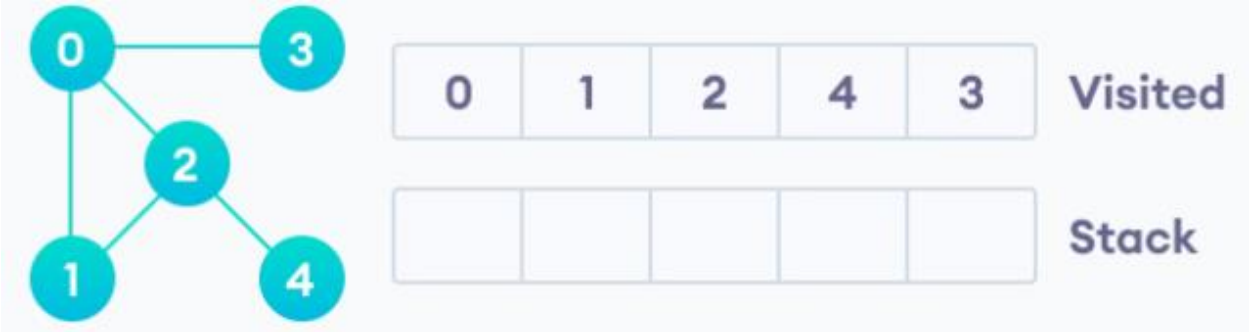


Vertex 2 has an unvisited adjacent vertex in 4, so we add that to the top of the stack and visit it.





After we visit the last element 3, it doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.



After we visit the last element 3, it doesn't have any unvisited adjacent nodes, so we have completed the Depth First Traversal of the graph.

Activity 7.3

- ✓ Explain the types of graph traversal?
- ✓ Explain the difference between BFS and DFS algorithms?

Chapter Seven Review Questions

1. What makes graph different from tree structures?
2. Discus the application of graph?
3. Discus weighted graph and its application?
4. Write a Java program that can implement BFS algorithm?
5. Write a Java program that can implement DFS algorithm?

Chapter Eight: Advanced Sorting and Searching Algorithms

8.1.Advanced Sorting

8.1.1. Shell sort

The shell sort, also known as the diminishing increment sort, was developed by Donald L. Shell in 1959. The idea behind shell sort is that it is faster to sort an array if parts of it are already sorted.The original array is first divided into a number of smaller subarrays, these subarrays are sorted, and then they are combined into the overall array and this is sorted.

Pseudo code for the Shell sort

```
Input an array of data
Divide data into n subarrays
for (i=1;i<=n;i++)
        sort subarray i
sort array data
```

How should the original array be divided into subarrays? One approach would be to divide the array into a number of subarrays consisting of contiguous elements (i.e. elements that are next to each other). For example, the array [abcdef] could be divided into the subarrays [abc] and [def]. However, shell sort uses a different approach: the subarrays are constructed by taking elements that are regularly spaced from each other. For example, a subarray may consist of every second element in an array, or every third element, etc. For example, dividing the array [abcdef] into two subarrays by taking every second element results in the subarrays [ace] and [bdf].

Actually, shell sort uses several iterations of this technique. First, a large number of subarrays, consisting of widely spaced elements, are sorted. Then, these subarrays are combined into the overall array, a new division is made into a smaller number of subarrays and these are sorted. In the next iteration a still smaller number of subarrays is sorted. This process continues until eventually only one subarray is sorted, the original array itself.

data before 5-sort	10	8	6	20	4	3	22	1	0	15	16
5 subarrays before 5-sort	10					3					16
		8					22				
			6					1			
				20					0		
					4					15	
5 subarrays after 5-sort	3					10					16
		8					22				
			1					6			
				0					20		
					4					15	
data after 5-sort and before 3-sort	3	8	1	0	4	10	22	6	20	15	16
3 subarrays before 3-sort	3			0			22			15	
		8			4			6			16
			1			10			20		
3 subarrays after 3-sort	0			3			15			22	
		4			6			8			16
			1			10			20		
data after 3-sort and before 1-sort	0	4	1	3	6	10	15	8	20	22	16
data after 1-sort	0	1	3	4	6	8	10	15	16	20	22

Fig. 8.1 Shell sort

In the first iteration, five subarrays are constructed by taking every fifth element. These subarrays are sorted (this phase is known as the *5-sort*). In the second iteration, three subarrays are constructed by taking every third element, and these arrays are sorted (the *3-sort*). In the final iteration, the overall array is sorted (the *1-sort*). Notice that the data after the 3-sort but before the 1-sort is almost correctly ordered, so the complexity of the final 1-sort is reduced.

In the above example, we used three iterations: a 5-sort, a 3-sort and a 1-sort. This sequence is known as the diminishing increment. But how do we decide on this sequence of increments? Unfortunately, there is no definitive answer to this question. In the original algorithm proposed by Donald L. Shell, powers of 2 were used for the increments, e.g. 16, 8, 4, 2, 1. However, this is not the most efficient technique. Experimental studies have shown that increments calculated according to the following conditions lead to better efficiency:

$n1 = 1$

$ni+1 = 3ni + 1$

For example, for a list of length 10,000 the sequence of increments would be 3280, 1093, 364, 121, 40, 13, 4, 1.

Another decision that must be made with shell sort is what sorting algorithms should be used to sort the subarrays at each iteration? A number of different choices have been made: for example, one technique is to use insertion sort for every iteration and bubble sort for the last iteration. Actually, whatever simple sorting algorithms are used for the different iterations, shell sort performs better than the simple sorting algorithms on their own. It may seem that shell sort should be less efficient, since it performs a number of different sorts, but remember that most of these sorts are on small arrays, and in most cases the data is already almost sorted. An experimental analysis has shown that the complexity of shell sort is approximately O(n^1.25), which is better than the O(n^2) offered by the simple algorithms.

Example2:

sort: (5, 8, 2, 4, 1, 3, 9, 7, 6, 0)

Solution:

4 – sort

Sort(5, 1, 6) – 1, 8, 2, 4, 5, 3, 9, 7, 6 , 0

Sort(8, 3, 0) – 1, 0, 2, 4, 5, 3, 9, 7, 6, 8

Sort(2, 9) – 1, 0, 2, 4, 5, 3, 9, 7, 6, 8

Sort(4,7) - 1, 0, 2, 4, 5, 3, 9, 7, 6, 8

1- sort

Sort(1, 0, 2, 4, 5, 3, 9, 7, 6, 8) – 0,1, 2, 3, 4, 5, 6, 7, 8, 9

Activity 8.1

- ✓ Explain how shell sort works?
- ✓ Explain what diminishing increment is and how to use it in shell sort?

8.1.2. Quick sort

Quick sort was developed by C. A. R. Hoare in 1961, and is probably the most famous and widely used of sorting algorithms. The guiding principle behind the algorithm is similar to that of shell sort: it is more efficient to sort a number of smaller subarrays than to sort one big array.

In quick sort the original array is first divided into two subarrays, the first of which contains only elements that are less than or equal to a chosen element, called the bound or pivot. The second sub-array contains elements that are greater than or equal to the bound. If each of these subarrays is sorted separately they can be combined into a final sorted array. To sort each of the sub-arrays, they are both subdivided again using two new bounds, making a total of 4 sub-arrays. The partitioning is repeated again for each of these subarrays, and so on until the sub-arrays consist of a single element each, and do not need to be sorted.

Quick sort is inherently recursive in nature, since it consists of the same simple operation (the partitioning) applied to successively smaller subarrays.

Pseudo code for the Quick sort

```
Input an array of data
If length of data > 1
Choose a bound
While there are elements left in data
Include element either in subarray1(if element <=bound) or in subarray2(if element
>=bound)
Quicksort(Subarray1)
Quicksort(Subarray2)
```

The pivot element can be selected in many ways. We may select the first element as a pivot, or we may select the middle element of the array or may be a randomly selected pivot [bound] element can be used. The following example demonstrates a pivot element selected from the middle of the data.

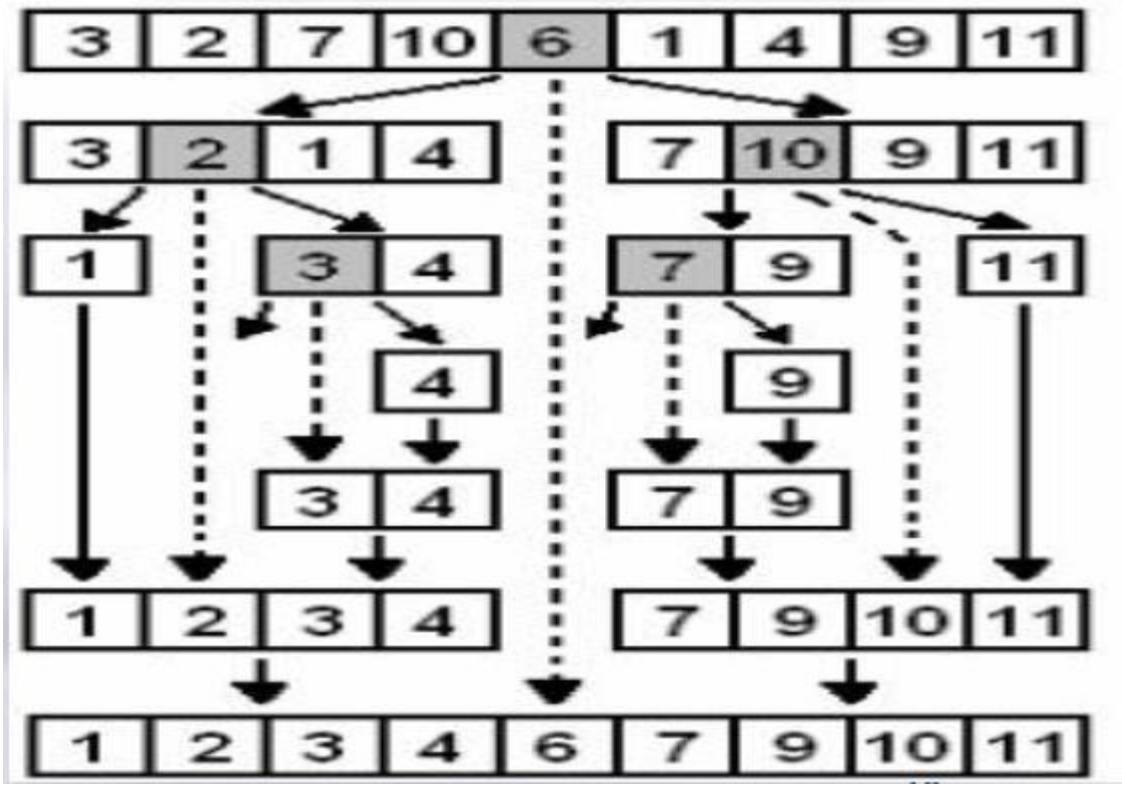


Fig. 8.2 Quick sort

To begin with, the middle element 6 is chosen as the bound, and the array partitioned into two subarrays. Each of these subarrays is then partitioned using the bounds 2 and 10 respectively. In the third phase, two of the four subarrays only have one element, so they are not partitioned further. The other two subarrays are partitioned once more, using the bounds 3 and 7. Finally we are left with only subarrays consisting of a single element. At this point, the subarrays and bounds are recombined successively, resulting in the sorted array.

8.1.3. Heap sort

Heap is a binary tree that has two properties:

- The value of each node is greater than or equal to the values stored in each of its children.
- The tree is perfectly balanced, and the leaves in the last level are all in the leftmost positions. (filled from left to right)

A property of the heap data structure is that the largest element is always at the root of the tree. A common way of implementing a heap is to use an array. The heap sort works by first rearranging

the input array so that it is a heap, and then removing the largest elements from the heap one by one.

Pseudo code for the heap sort

```
Transform data into a heap
for (i = n-1; i >=1; i--)
    Swap the root with element in position i
    Restore the heap property for the tree
```

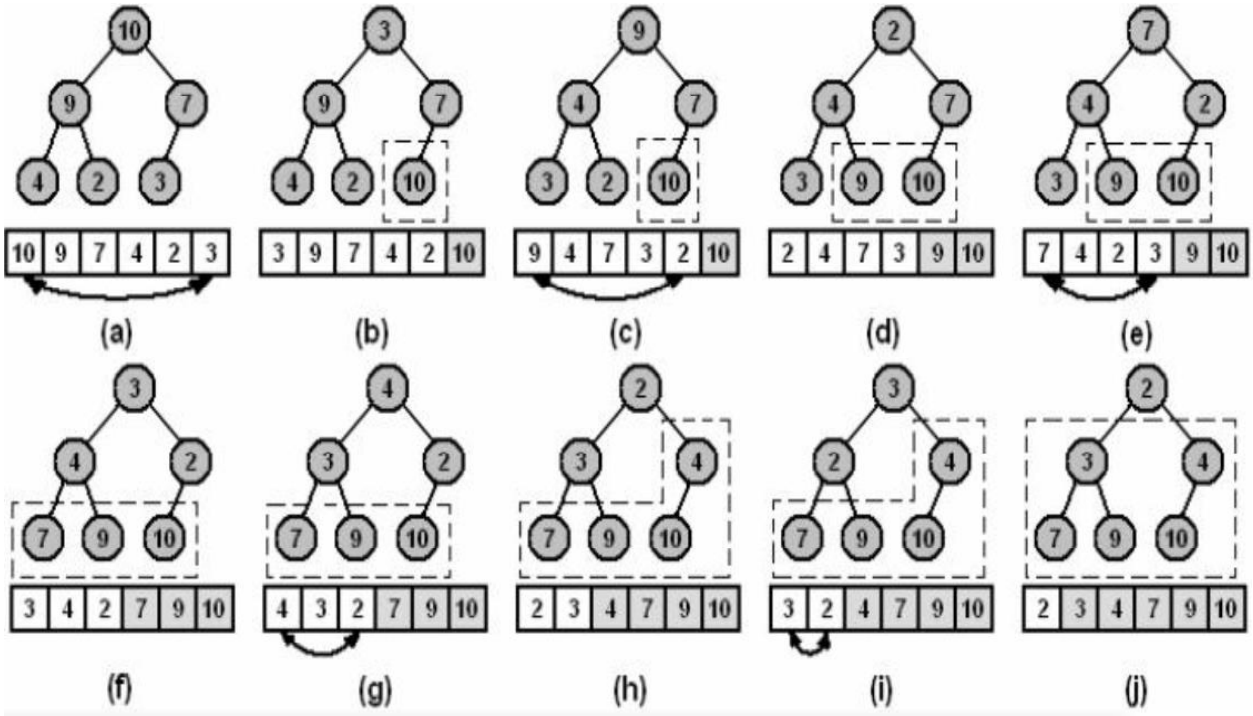


Fig. 8.3 Heap sort

Activity 8.2

- ✓ Explain how the heap sort works?
- ✓ Explain how quick sort works?

8.1.4. Merge sort

Merge sort also works by successively partitioning the array into two sub-arrays, but it guarantees that the sub-arrays are of approximately equal size. This is possible because in mergesort the array is partitioned without regard to the values of their elements: it is simply divided down the middle into two halves. Each of these halves is recursively sorted using the same algorithm. After the two sub-arrays have been sorted, they are merged back together again. The recursion continues until the sub-arrays consist of a single element, in which case they are already sorted.

Pseudo code for the Merge sort

Input an array of data

If length of data > 1

Mergesort(left half of data)

Mergesort(right half of data)

Merge the left and right halves back together

The following example demonstrates how merge sort divide the problem and combine the solution.

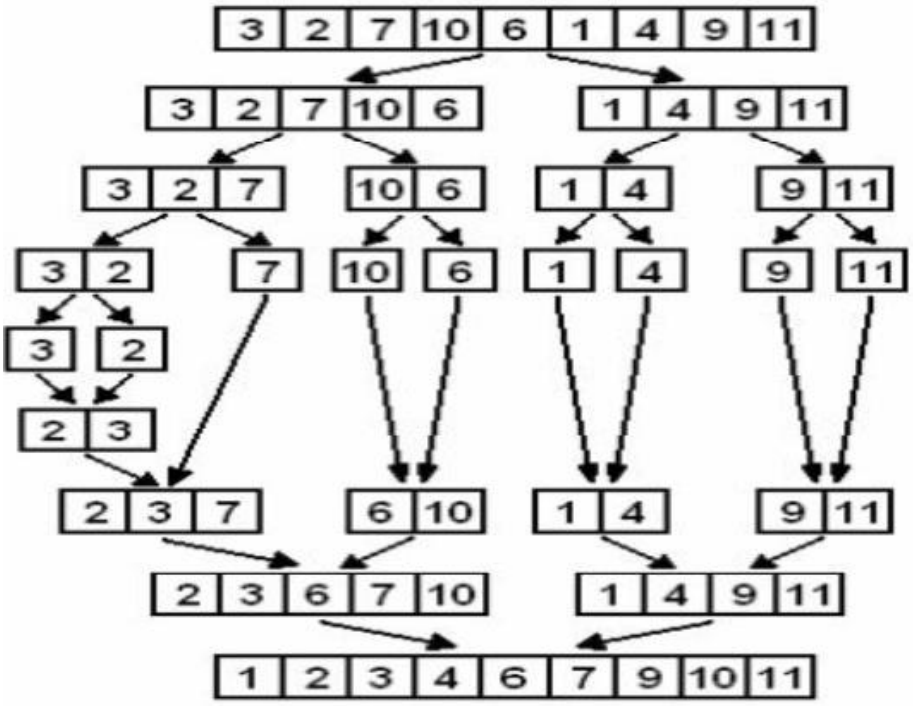


Fig. 8.4 Merge sort

8.2.Advanced Searching

8.2.1. Hashing

Assume a company has 100 employees and each of the 100 employees has an ID number in the range 0 to 99 and we want to access the employee records using the key IDNumber. If we store the elements in an array that is indexed from 0 to 99, we can directly access any employee's record through the array index. Consider the company uses its employees five-digit IDNumber as the primary key, now the range of the key values goes from 00000-99999. Obviously, it is impractical to set up an array of 100,000 elements of which only 100 are needed. What if we use just the last two digits of the key to identify each employee? For example 54464 is in [64], and 83245 is in [45].

Hashing: the technique used for ordering and accessing elements in a list in a relatively constant amount of time by manipulating the key to identify its location in the list. In the case of the employee list, the hash function is (key%100). The key(IDNumber) is divided by 100, and the remainder is used as an index into the array of employee elements.

```
Void hash()
        {
        return (IDNumber%Maximum_Size);
        }
```

Example key=50704 (key%100) => it is at index 4

Activity 8.3

- ✓ Explain how the merge sort works?
- ✓ Explain what hashing is?

Chapter Eight Review Questions

1. How to divide the original array into subarrays using shell sort?
2. What are the properties of heap?
3. How to divide the original array into subarrays using quick sort?
4. What is the difference between quick and merge sorts when dividing into subarrays?
5. Explain the application of hashing?